Reference: 2004/08/05/orabpel/d5/1.1

Oracle BPEL Process Manager 2.0
Test Drive

This document describes how BPEL and the Oracle BPEL Process Manager facilitate development of SOA based applications by composing a set of synchronous and asynchronous services into an end-to-end BPEL process flow. It is not intended to be a complete development guide, but rather a tutorial and guided tour providing a rapid overview of many of the most commonly used features. Pointers are given throughout to additional documentation and samples and readers should also use the many other documents available at http://otn.oracle.com/bpel after going through this guided tour.

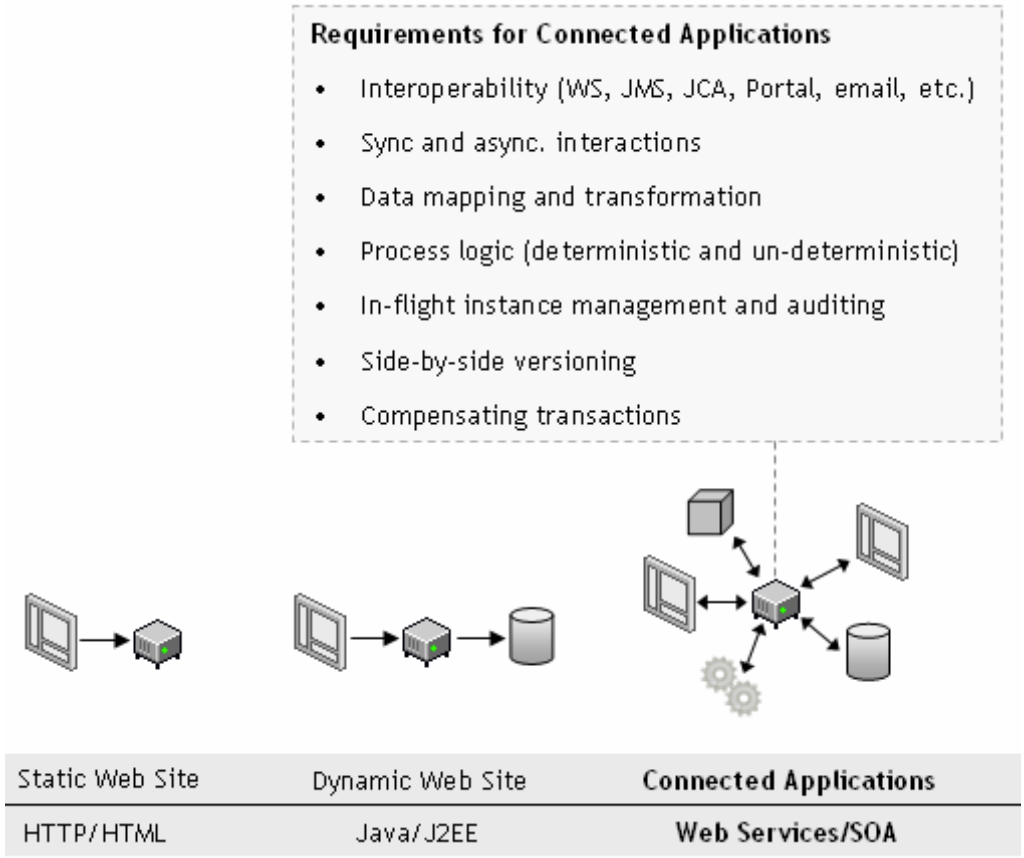
Note: Much of this document is a tutorial with instructions for you to work hands-on with a local installation of the Oracle BPEL Process Manager. However, you can also just use this document to get some of the "test drive experience" even if you do not plan to install our server at this time.

Contents

BPEL, the Cornerstone of SOA

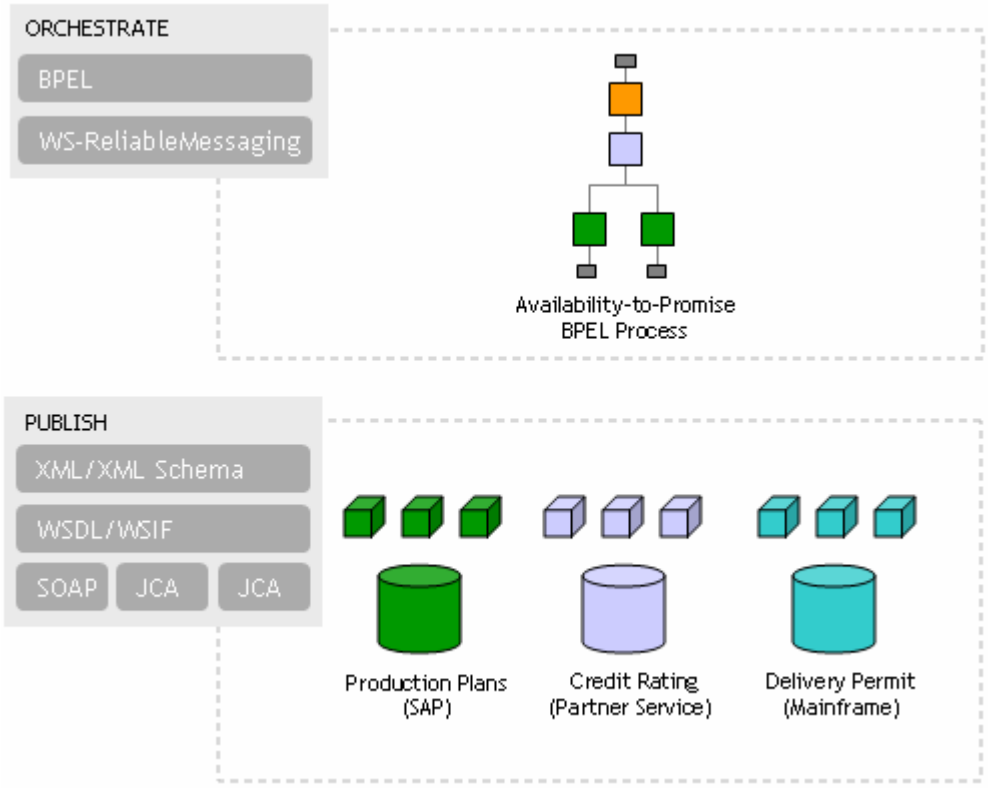
TOWARDS CONNECTED APPLICATIONS

An increasing number of companies are looking at Web services and SOA as an architectural blue-print and a set of standards for addressing the integration requirements involved in building connected applications. While SOA has been a best practice for over a decade, there was some confusion about which interfaces to adopt. BPEL and Web services standards have solved this problem by addressing common application requirements in an open, portable and standard way. SOA enables business agility by maximizing leverage of existing resources while minimizing the cost of deploying new applications. Enterprises adopting these standards and architectural approach are already achieving significant ROI from using the same standards-based approach to building connected applications that they have used for building web applications with Java/J2EE.



Making web services work is a 2-step process. First you publish your services and then you compose, or orchestrate, them into business flows. Publishing a service means taking a function within an existing application or system and making it available in a standard way, while orchestration is composing multiple services into an end-to-end business process. The Web services standards, including WSDL, XML and SOAP, have emerged as an effective and highly interoperable platform for publishing services. In addition, high

performance binding frameworks are allowing enterprises to access legacy systems and native Java code without necessarily having to wrap them in a SOAP interface.



KEY BPEL CONCEPTS

BPEL (Business Process Execution Language) is emerging as the clear standard for composing multiple synchronous and asynchronous services into collaborative and transactional process flows. BPEL benefits from 15+ years of research poured into its predecessor languages, XLANG and WSFL.

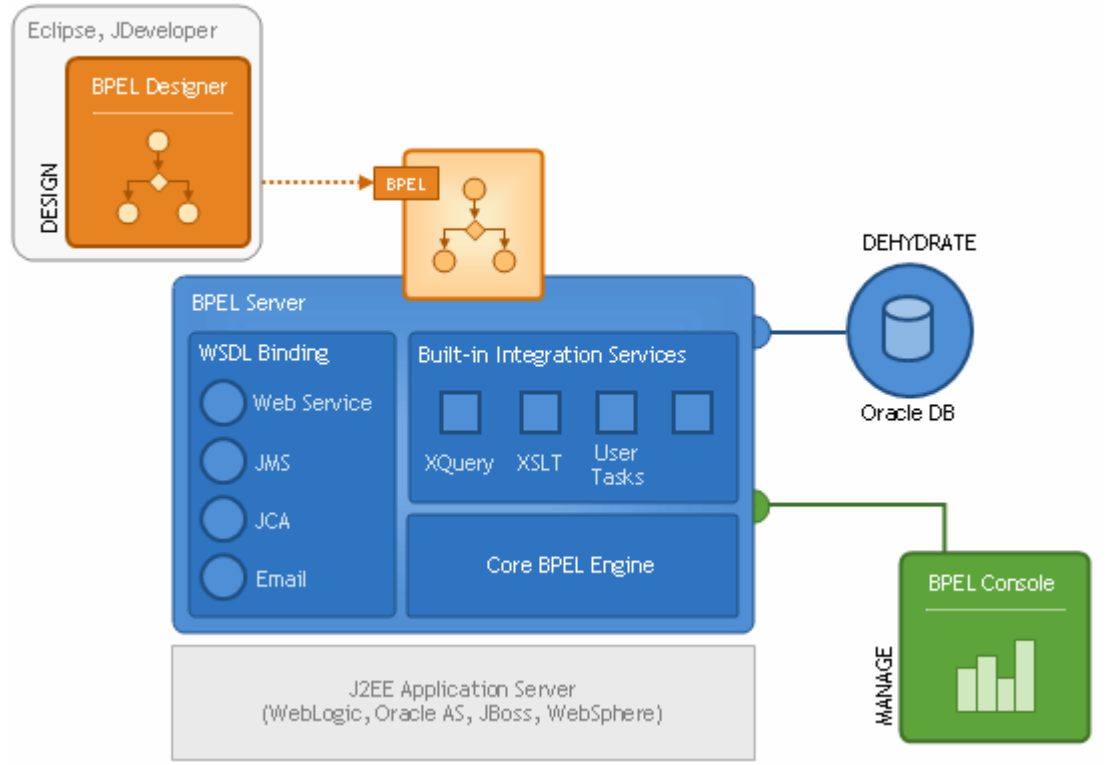
- Web Services/WSDL as component model
- XML as data model (data loose-coupling)
- Sync. and async. message exchange patterns
- Deterministic and non-deterministic flow coordination
- Hierarchical exception management
- Long-running unit of work/compensation

Since the BPEL specification was submitted to OASIS in March 2003, it has gained the support of nearly every major industry vendor (including Oracle, Microsoft, IBM, SAP, Siebel, BEA and Sun). With the list of vendors committing to support for BPEL growing every day and several BPEL server implementations publicly available, the standards war in the business process space appears to be over. And this is a great thing for enterprises who can now implement their business processes in a standard and portable way, avoiding vendor lock-in to a degree that has not been possible before.

The Oracle BPEL Process Manager

INTRODUCTION

The Oracle BPEL Process Manager provides a developer-friendly and reliable solution for designing, deploying and managing BPEL business processes.



The **BPEL Designer** provides a graphical and user-friendly way to build BPEL processes. What is unique about the Oracle BPEL Designer is that it uses BPEL as its native format. This means that processes built with the Designer are 100% portable and in addition it enables developers to view, and modify, the BPEL source without decreasing the usefulness of the tool.

The **core BPEL engine** provides the most mature, scalable and robust implementation of a BPEL server available today. The Oracle BPEL Process Manager executes standard BPEL processes and provides a "dehydration" capability so that the state of long-running flows is automatically maintained in a database, enabling clustering for both fail-over and scalability. The BPEL Server leverages an underlying J2EE application server, with support for most major commercial application servers and a bundled version available.

The **built-in integration services** enable developers to easily leverage advanced connectivity and transformation capabilities from standard BPEL processes. These capabilities include support for XSLT and XQuery transformation as well as bindings to hundreds of legacy systems through JCA adapters and native protocols. A user task

service is provided as a built-in BPEL service to enable the integration of people and manual tasks into BPEL flows.

The **extensible WSDL binding framework** enables connectivity to protocols and message formats other than SOAP. Bindings are available for JMS, email, JCA, HTTP GET and POST and many other protocols enabling simple connectivity to hundreds of back-end systems.

The **BPEL Console** provides a mature web-based interface for management, administration and debugging of processes deployed to the BPEL server. Audit trails and process history/reporting information is automatically maintained and available both through the BPEL Console and via a Java API.

HOW IS IT DIFFERENT?

The power of an open standard - By capturing your business processes in BPEL, you protect your intellectual property and investments while avoiding vendor lock-in. BPEL is to business process management what SQL was to data management.

Unparalleled visibility and administration - The BPEL Console reduces the cost and complexity of deploying and managing your business processes: Visually monitor the execution of each BPEL process, drill down into the audit and view the details of each conversation or debug a running flow against its BPEL implementation

Dramatic cost savings - The Oracle BPEL Process Manager is 60-80% less expensive to buy and maintain than traditional EAI solutions.

Open and flexible binding framework - Orchestrate XML Web services, but also Java/J2EE components, portals, JCA interfaces and JMS destinations. Tie into 100+ back end systems. Leverage your Java skills and application server investments.

FEATURE SUMMARY

BPEL DESIGNER

- Native BPEL Support
- Drag-and-drop process modeler
- UDDI and WSIL service browser
- Visual XPATH editor
- One-click build and deploy

BUILT-IN INTEGRATION SERVICES

- Java embedding
- Email and JMS messaging services
- XSLT and XQuery transformation services
- User task manager and portal integration
- Extensible WSIF binding framework

BPEL CONSOLE

- Visual Monitoring
- Auditing
- BPEL Debugging
- In-flight Administration
- Performance Tuninng
- Partitioning/Domains

BPEL SERVER

- Comprehensive BPEL 1.1
- Sync. And async messaging
- Context Dehydration
- Advanced Exception Management
- Side-by-side versioning
- Large XML documents

Installation Instructions

Theses instructions are based on version 0.9 of the BPEL Designer and the 2.0 RC9 release of the Oracle BPEL Process Manager.

PREREQUISITES

- Internet Explorer 6.0
- JDK 1.4.1 (or later)
- Windows 2000 or XP, 384 MB RAM
- 100MB of disk space for the Eclipse 3.0 GA, 25 MB of disk space for the BPEL Designer plug-in and 100MB of disk space for the BPEL Server
- Some familiarity with XML Schema, WSDL, XPath, BPEL, and related Web service standards

DOWNLOAD AND INSTALL

The BPEL Designer is an Eclipse (http://www.eclipse.org) plug-in supporting version 3.0GA so you will first need to download and install a compatible Eclipse distribution. The Designer should work with later versions of Eclipse as well, but these instructions have been tested with the 3.0GA release and are supported for that release only.

1 Make sure you have a 1.4.1 (or later) JDK. If you don't have one, the latest JDK for Windows can be downloaded from Sun at: http://javashoplm.sun.com/ECom/docs/Welcome.jsp?StoreId=22&PartDetailId=j2sdk-1.4.2_04-oth-JPR&SiteId=JSC&TransactionId=noreg.

2 Download the Eclipse 3.0 GA distribution. A Microsoft Windows version of 3.0 GA can be gotten from the eclipse download site: http://www.eclipse.org/downloads/index.php

3 Unzip your Eclipse distribution *somewhere*. The rest of this document assumes you unzip it into C:\ directory on Windows, so you should adjust paths as appropriate if you place it somewhere else.

4 Run your Oracle BPEL Designer installer executable (if you don't have one already, it can be downloaded from http://otn.oracle.com/bpel). Note that the installer will check that you have an appropriate base version of Eclipse and a JDK. The rest of this document assumes the default BPEL Server installation directory of: `C:\orabpel`.

5 Run your Oracle BPEL Process Manager server installer executable (if you don't have one already, it can be downloaded from http://otn.oracle.com/bpel). Note that the installer will check that you have an appropriate base version of Eclipse and a JDK. The rest of this document assumes the default BPEL Server installation directory of: `C:\orabpel`.

Now you will be able to run your Oracle BPEL Process Manager and BPEL Designer, as shown below.

To start the BPEL Designer:

6 Start the BPEL Designer from the Windows Start menu by clicking **Start > Programs > Oracle > BPEL Process Manager 2.0 > BPEL Designer**.

Alternatively, you can double-click `C:\orabpel\bin\bpelz.bat` in the Windows Explorer.

To start the BPEL Server:

You should also start your Oracle BPEL Server, since you will be using it to deploy and test flows that you create with the Designer.

7 Start the Oracle BPEL Process Manager from the Windows Start menu by clicking **Start > Programs > Oracle > BPEL Process Manager 2.0 > Start BPEL Process Manager**.

Alternatively, if you will use the Designer without the Oracle BPEL Process Manager installed locally, see the Knowledge Base article at http://otn.oracle.com/bpel for instructions on how to run the Designer stand-alone.

Example I: Your First BPEL Process

In this section of the guided tour, you will learn how to use the Oracle BPEL Designer to build, deploy, and test your first BPEL process. The process is an asynchronous flow that calls a simple service: a synchronous credit rating service. Creating this process is intended to be the first step toward building a more sophisticated application (like the loan flow example).

GETTING STARTED

Before starting the main content of this tutorial, you should compile and deploy the credit rating service that you will invoke as part of the tutorial, making it available on your local BPEL Server. You will use the command line rather than the BPEL Designer to compile and deploy this service.

To compile and deploy the credit rating service:

1 Open up a command prompt if you do not already have one open.

2 Change the directory to `C:\orabpel\samples\utils\CreditRatingService`, as follows:

```
> cd C:\orabpel\samples\utils\CreditRatingService
```

3 Execute the **obant** command.

```
> obant
```

To test the credit rating service:

4 If you have not already done so, connect to the BPEL Console (at http://localhost:9700/BPELConsole) and log in.

5 In the **Name** column (under the **Dashboard** tab of the console), click the link for the `CreditRatingService` BPEL process.

6 In the test form that appears, enter any nine-digit number as the social security number (for example, 123456789) and click **Post XML Message**.

You should get back an integer credit rating (or a fault if the social security number you entered began with a 0). In either case, the service is confirmed to be installed successfully.

CREATE A NEW BPEL PROJECT

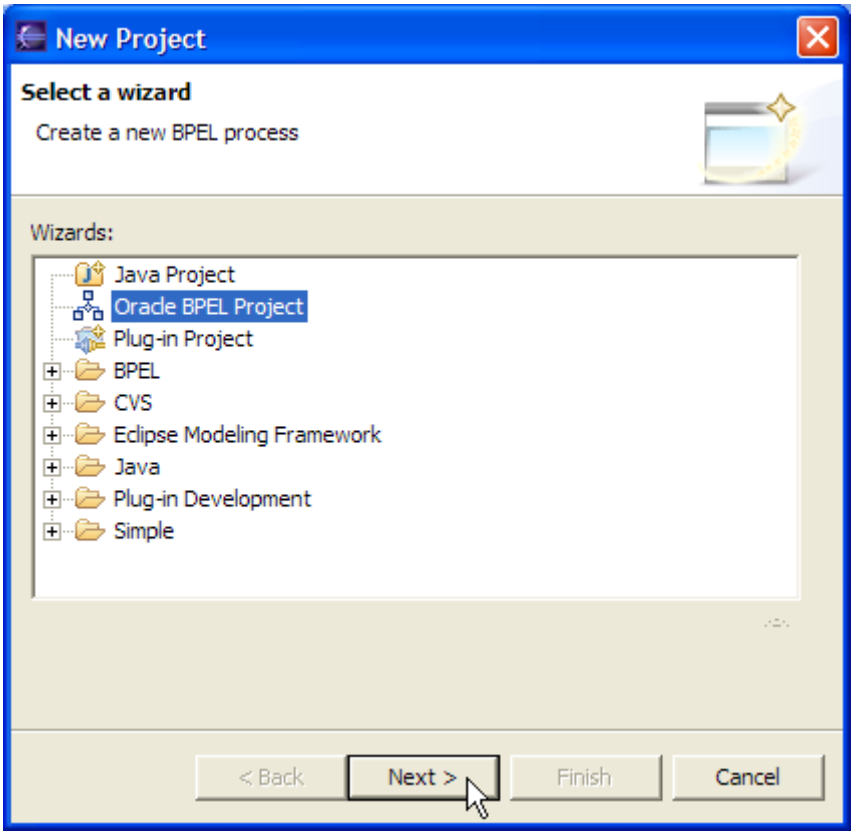
You will use the BPEL Designer's New Project wizard, which automatically generates the skeleton of a BPEL project: the BPEL source, a WSDL interface, a BPEL deployment descriptor, and an Ant script for compiling and deploying the BPEL process.

To create a new BPEL project:

1 In the BPEL Designer, click **File** > **New** > **Project**.

Select **Oracle BPEL Project**.

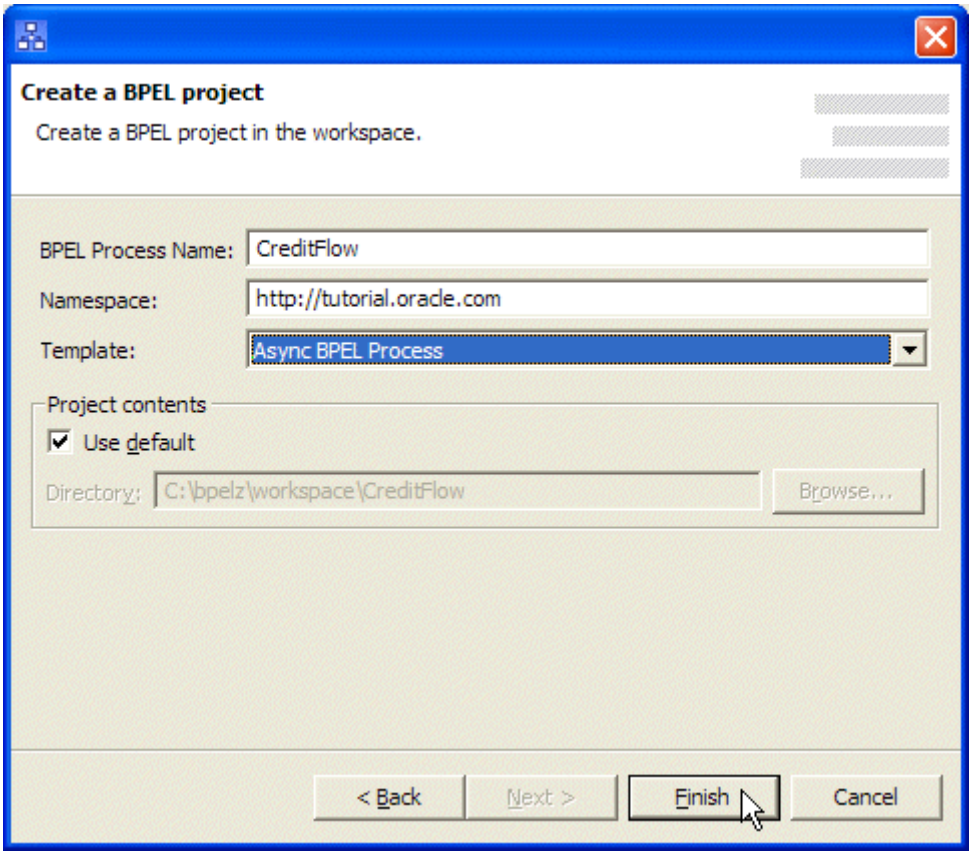
2 Click **Next**.



3 Enter a BPEL process name of `CreditFlow`.

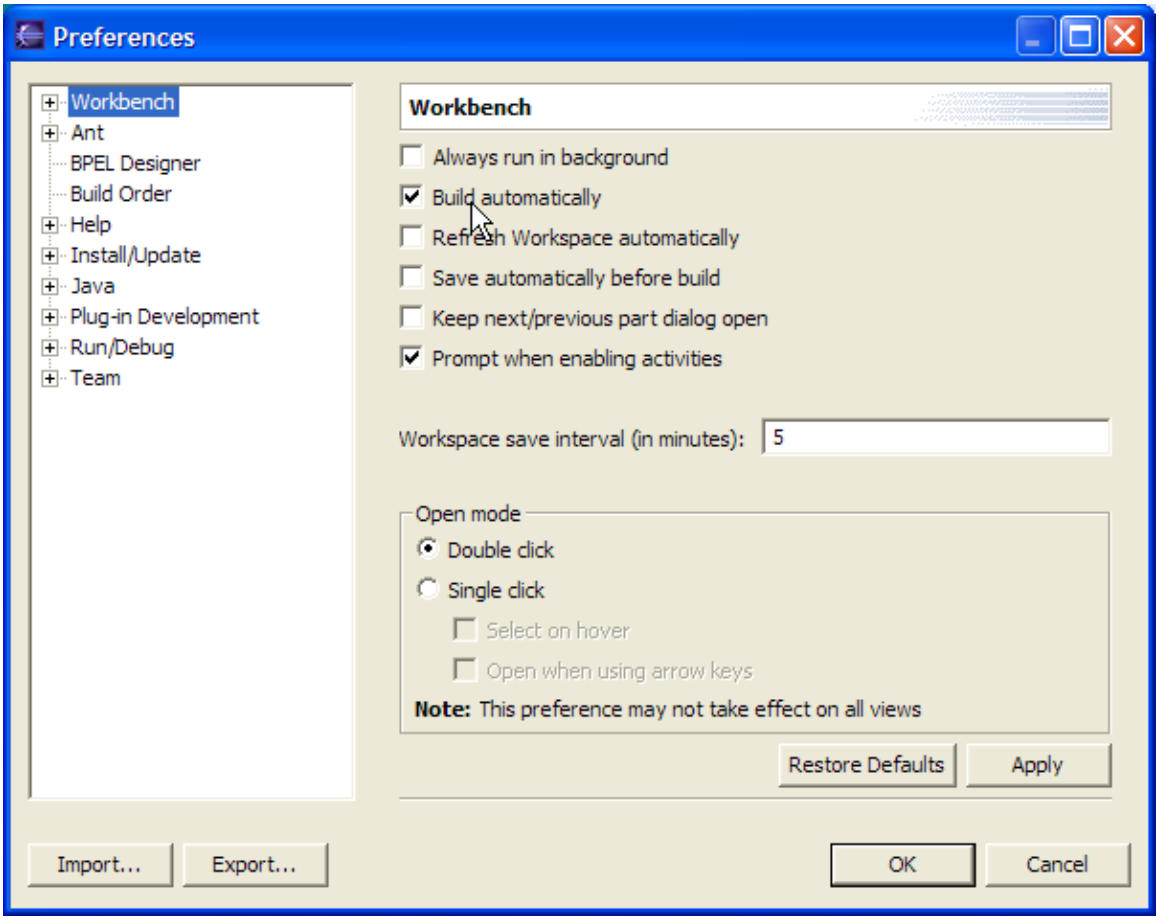
4 [Optional] Enter `http://tutorial.oracle.com` as the namespace.

5 Leaving the template as **Async BPEL Process** and the **Use default** location at its default setting, click **Finish**.



The new project will be created in the `C:\eclipse\workspace` directory. You can see (in the Navigator within the BPEL Designer) that the New Project wizard has created a skeleton for a new asynchronous BPEL process, with all the necessary source files. The file names are the same whether you create a synchronous or an asynchronous process, but the contents of the `.bpel` and `.wsdl` files will differ as appropriate. If you would like to go through a tutorial of creating a synchronous BPEL process, a simple HelloWorld tutorial is available at http://otn.oracle.com/bpel which shows how to create a synchronous BPEL process with the Designer.

Note that Eclipse is initially configured with an option that will automatically compile any new projects that you create or import, so you may notice a console window at the bottom of your Designer window with compilation messages. This is not always what you want, but you can easily disable this option by selecting the **Window > Preferences** menu, then clicking the Workbench tab and turning off the "Build automatically" preference:



REVIEW THE WSDL INTERFACE OF YOUR ASYNCHRONOUS BPEL PROCESS

You will now review (and, in the next section, edit) the WSDL file for the process.

To view the WSDL interface:

1 In the Navigator, double-click the `CreditFlow.wsdl` file to edit it.

2 Scroll down as necessary to browse the code. The most "interesting" parts are pointed out below.

Notice that the New Project wizard has defined both a `CreditFlowRequest complexType` element that your flow accepts as input (in a document-literal style WSDL message) and a `CreditFlowResponse` element that it returns.

Because this process is asynchronous, two portTypes are defined, each with a one-way operation: one to initiate the asynchronous process and one for the process to call back the client with the asynchronous response.

```
<!-- portType implemented by the CreditFlow BPEL process -->
<portType name="CreditFlow">
    <operation name="initiate">
        <input message ="tns:CreditFlowRequestMessage"/>
    </operation>
</portType>

<!-- portType implemented by the requester of CreditFlow BPEL process
     for asynchronous callback purposes -->
<portType name="CreditFlowCallback">
    <operation name="onResult">
        <input message ="tns:CreditFlowResponseMessage"/>
    </operation>
</portType>
```

Also note that the partnerLinkType defined for this asynchronous process has two roles, one for the service provider and one for the requester.

```
<plnk:partnerLinkType name="CreditFlow">
    <plnk:role name="CreditFlowProvider">
        <plnk:portType name="tns:CreditFlow"/>
    </plnk:role>
    <plnk:role name="CreditFlowRequester">
        <plnk:portType name="tns:CreditFlowCallback"/>
    </plnk:role>
</plnk:partnerLinkType>
```

EDIT THE WSDL INTERFACE OF YOUR BPEL PROCESS

Now you will edit the WSDL file to modify the input and output messages.

To modify the input and output messages of your BPEL process:

1 While still editing your `CreditFlow.wsdl` file, change the two type definitions so that your flow will take an `ssn` field as input (keeping the `string` type) and return a `creditRating` element as output of type `int`. The parts you need to change are shown in bold (and red) below.

```
<element name="CreditFlowRequest">
    <complexType>
        <sequence>
            <element name="ssn" type="string">
        </sequence>
    </complexType>
</element>

<element name="CreditFlowResponse">
    <complexType>
        <sequence>
            <element name="creditRating" type="int">
        </sequence>
    </complexType>
</element>
```

2 Save your WSDL file and close the window in which you have been editing it.

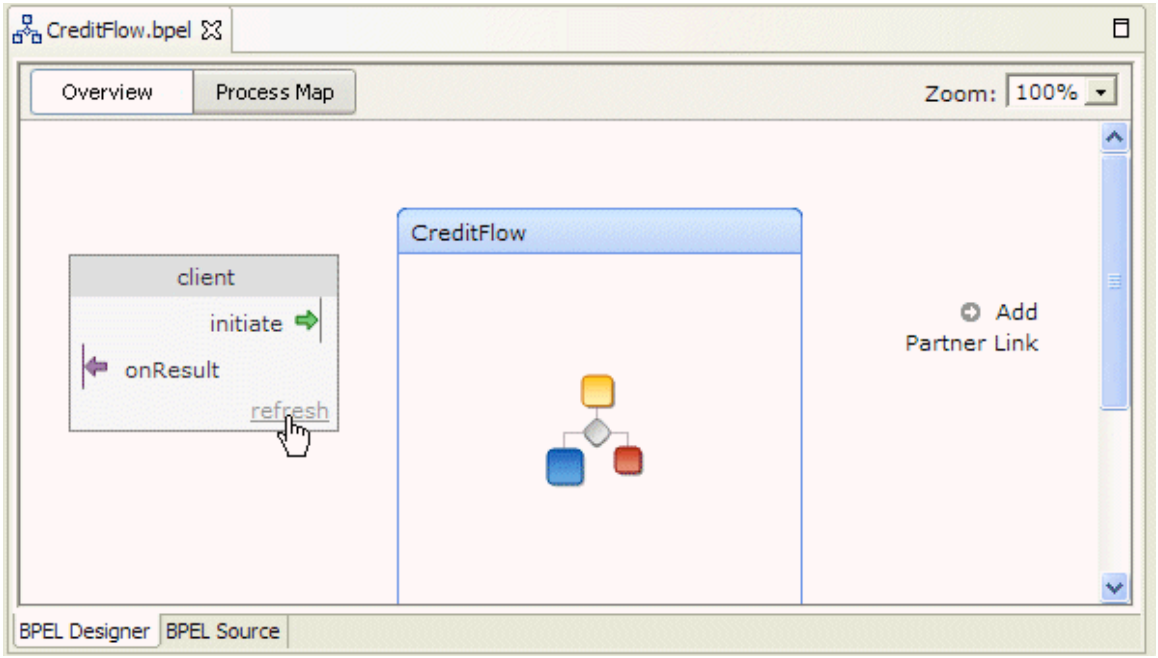
SWITCH BETWEEN THE OVERVIEW, PROCESS MAP, AND SOURCE CODE

You will now turn to the BPEL file and view it in a variety of ways.

To view an overview of the BPEL process:

3 Double-click `CreditFlow.bpel` in the Navigator if it is not already open in the Designer (however, it will open by default when you create a new BPEL project so it should be already open if you have followed this tutorial exactly).

The initial view that you will see for editing a BPEL process file is a visual representation providing an overview of your process. On the left is the client interface, displaying operations exposed by the process (`initiate` in this case) and any asynchronous callback operations (`onResult`). Note that the BPEL Designer automatically puts a partnerLink named `client` on the left side of the window. All other partnerLinks will be displayed in a list on the right.



4 Because you have changed your WSDL/client interface to your flow, you will need to refresh the client interface to your BPEL process, if you currently have the CreditFlow.bpel process open in the BPEL Designer. To do this, click the “refresh” link in the client interface as shown above.

REVIEW THE BPEL SOURCE CODE

The New Project wizard has created a basic skeleton for you of an asynchronous BPEL process.

To view the BPEL source code:

1 With `CreditFlow.bpel` open and active, click the **BPEL Source** tab at the bottom of the `CreditFlow.bpel` window.

2 Scroll down as necessary to browse the code. The interesting parts are pointed out below.

The partnerLink created for the client interface includes two roles, `myRole` and `partnerRole` assignment. As you saw in the WSDL file for this process, an

asynchronous BPEL process typically has two roles for the client interface: one for the flow itself, which exposes an input operation, and one for the client, which will get called back asynchronously.

```
<partnerLinks>
    <!-- comments... -->
    <partnerLink name="client"
                 partnerLinkType="tns:CreditFlow"
                 myRole="CreditFlowProvider"
                 partnerRole="CreditFlowRequester"
                 />
</partnerLinks>
```

Also, the `<receive>` activity in the main body of the process is followed by an `<invoke>` activity to perform an asynchronous callback to the requester. (Note the difference between this and a synchronous process, which would use a `<reply>` activity to respond synchronously to the caller.)

```
<sequence name="main">

    <!-- Receive input from requester.
         ...
         -->
    <receive name="receiveInput" partnerLink="client"
             portType="tns:CreditFlow"
             operation="initiate" variable="input"
             createInstance="yes"/>

    <!-- Asynchronous callback to the requester.
         ...
         -->
    <invoke name="callbackClient"
            partnerLink="client"
            portType="tns:CreditFlowCallback"
            operation="onResult"
            inputVariable="output"
            />
</sequence>
```

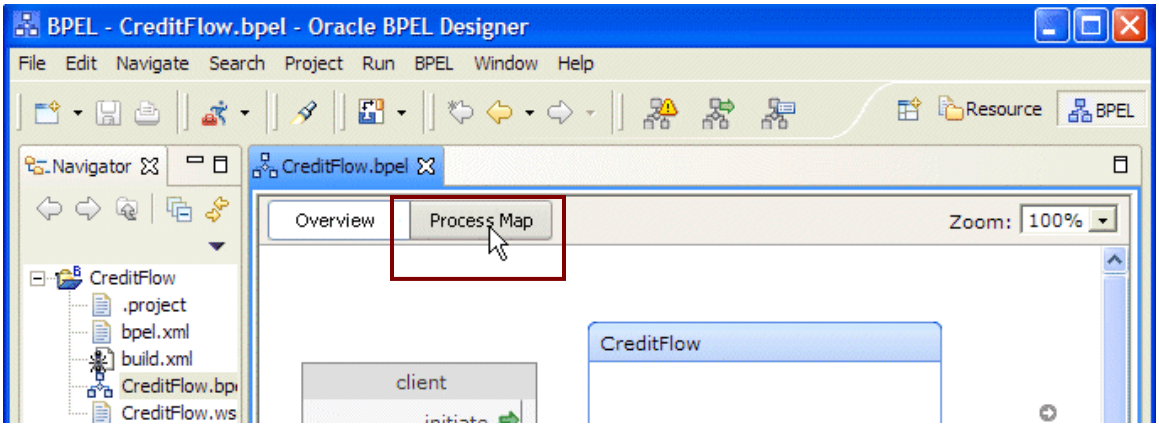
VIEW THE PROCESS MAP

You will now go to the Process Map view, where you will edit your BPEL process.

1 Click the **BPEL Designer** tab at the bottom of the window.

This returns you to the Overview view of the process. Here you can see in the client interface, that the input operation generated by the New Process wizard is named `initiate` and there is an asynchronous callback operation named `onResult`.

2 Click the **Process Map** button at the top of the window (or the **Edit Process Map** link in the middle of the window).



ADD ACTIVITIES TO THE PROCESS MAP

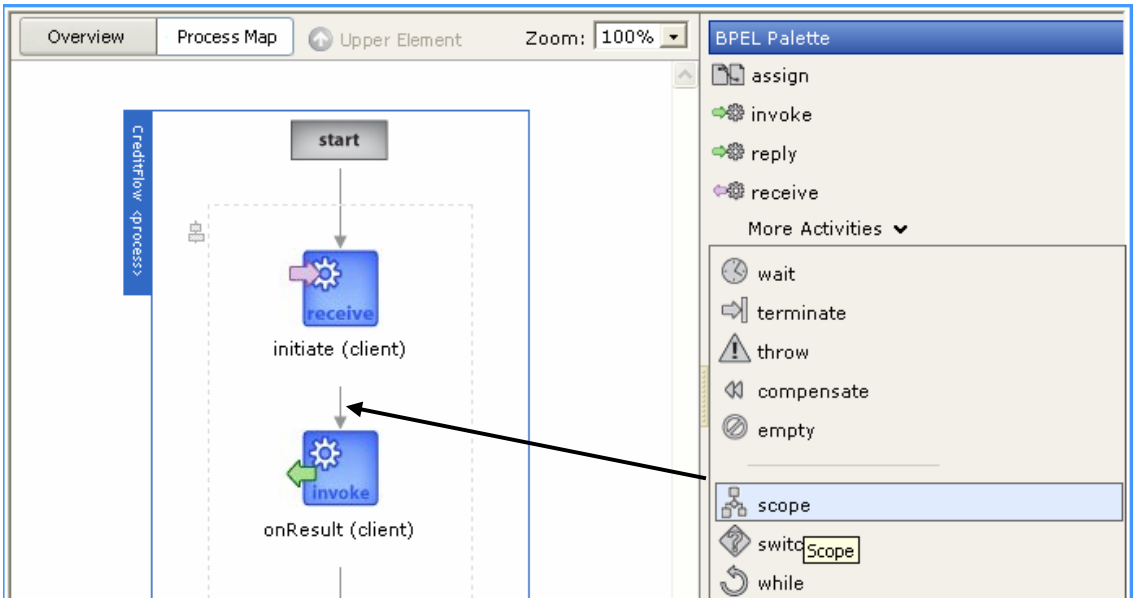
You are now ready to edit the process. You will start by adding two new activities to it: a `<scope>` activity and an `<invoke>` activity.

To insert a <scope> activity:

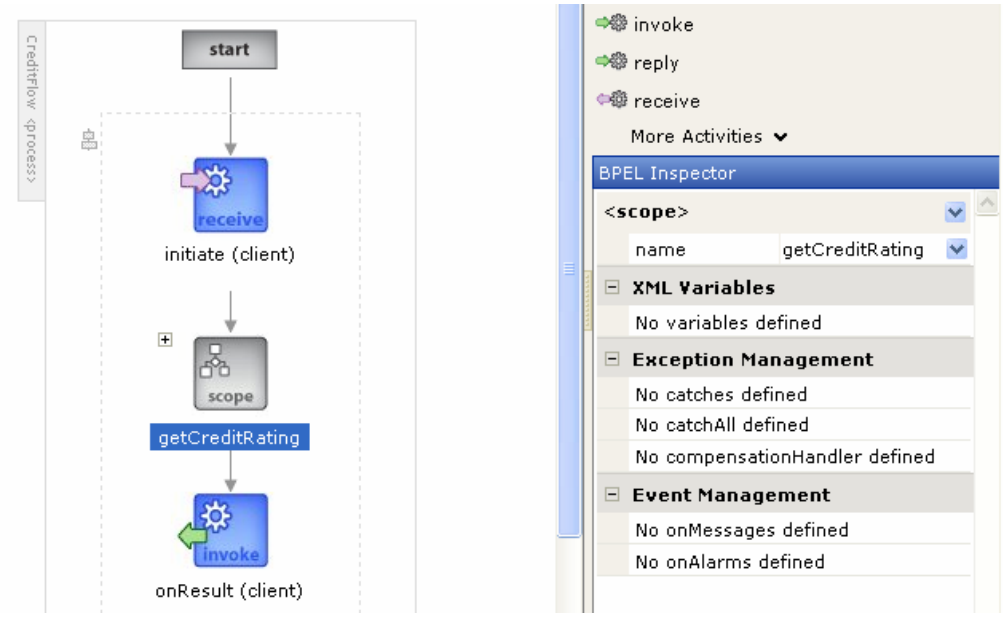
A BPEL scope is a collection of activities that can have its own local variables, exception handling, compensation, and so on — very much analogous to a programming language `{ }` block.

1 In the BPEL Palette on the right, click the **More Activities** link to see additional BPEL activities.

2 Drag a `<scope>` activity (from **scope** on the BPEL Palette) to the transition arrow between the `initiate (client) <receive>` activity and the `onResult (client) <invoke>` callback element.



3 In the BPEL Inspector, enter the value `getCreditRating` for the `name` attribute of the newly created scope.



To insert an <invoke> activity into the scope:

4 Click the "+" icon to the left of the `<scope>` activity in the process flow, to expand the scope so that you can drop activities into it.

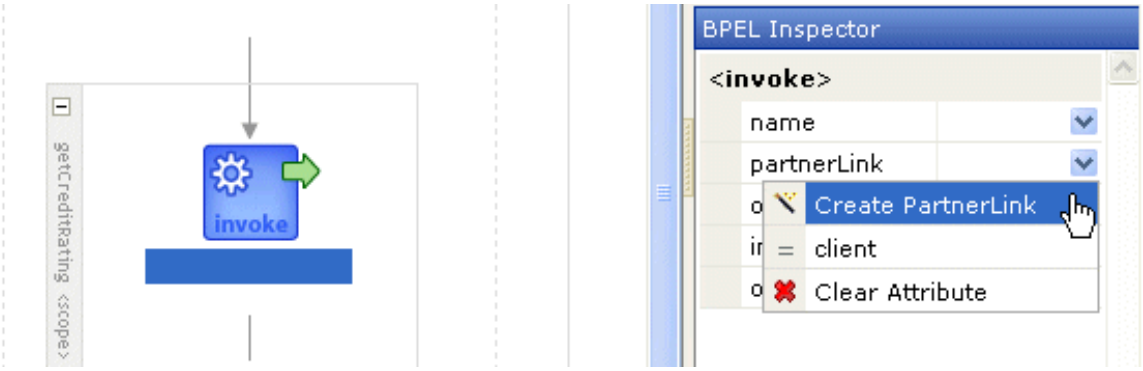
5 Drag an `<invoke>` activity from the BPEL Palette into the **Drop activity here** area within the scope.

The next step is to configure the `<invoke>` activity to call your intended service (in this case the credit rating service). In BPEL, this means you first need to create a partnerLink for the service.

CREATE A PARTNERLINK FOR THE CREDIT RATING SERVICE

In the BPEL Designer, you can add partnerLinks to your BPEL process either in the Overview view or at the time you configure the `<invoke>` activity (using a wizard). Here you will use the latter approach to add a partnerLink for the credit rating service that you built and deployed locally at the beginning of this tutorial.

1 In the BPEL Inspector, select **Create PartnerLink** in the drop-down list for the `partnerLink` attribute.



This will bring up the New partnerLink Wizard.

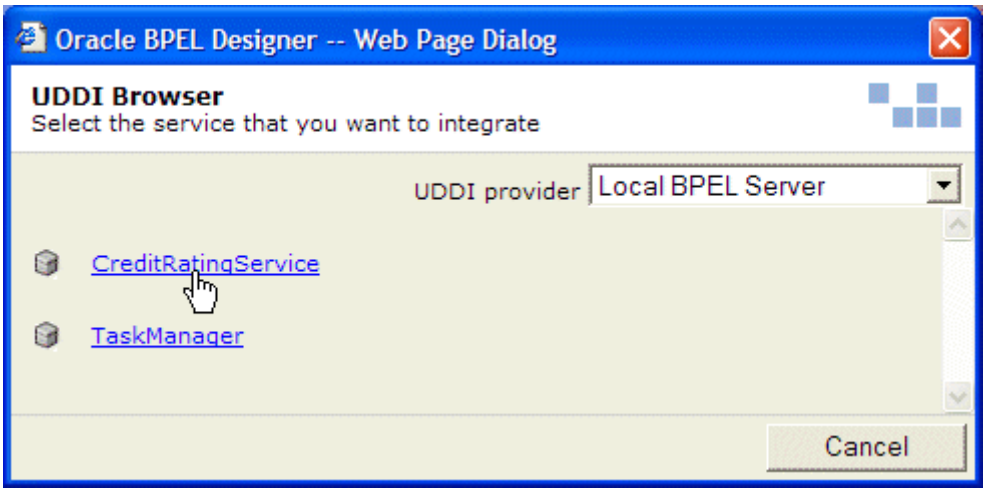
2 Enter the name `creditRatingService` for the partnerLink.

3 Enter the location of the WSDL file as follows:

a. Click the "..." button to the right of the `wsdlLocation` field.

The wizard presents a UDDI browser listing all the services deployed on the local Oracle BPEL server. (It may take a while for the browser to come up the first time.)

b. In the UDDI browser, click `CreditRatingService`.



The browser will close and the URL of the service will appear in the `wsdlLocation` field. The wizard will fetch the contents of the WSDL file so that it can populate drop-down lists appropriately.

Alternatively, you can enter the URL for a WSDL directly in this field if you know the specific location of the WSDL for the service you want to invoke. If you are using a commercial UDDI server, please submit a support request at http://otn.oracle.com/bpel for information on how to register your UDDI server with the BPEL Designer.

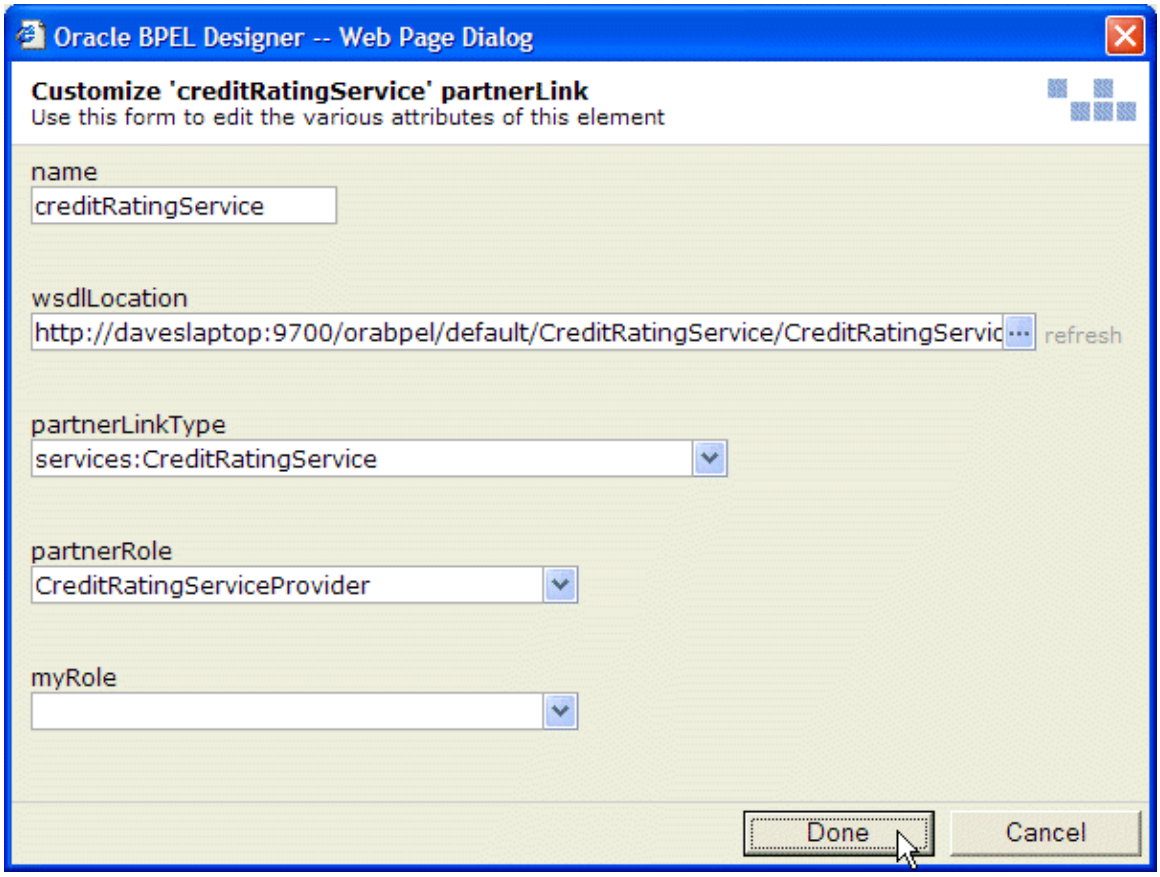
4 In the `partnerLinkType` list, select `services:CreditRatingService` (the only one defined in the WSDL file).

Note: If your service's WSDL file does not include a `partnerLinkType` definition, the BPEL Designer will issue a message to this effect, and you will need to add a `partnerLinkType` definition to the WSDL file. A dialog will prompt you as to whether you want the BPEL Designer to automatically generate a WSDL wrapper which adds the `partnerLinkType` for you in this case.

5 In the `partnerRole` list, select `CreditRatingServiceProvider`.

You will leave the `myRole` field blank. (Because this is a synchronous service without any callbacks, the client does not need a role.)

6 Click **Done**.



A new partnerLink will be added to your flow (automatically creating namespace shortcuts in your BPEL file as appropriate), and the `<invoke>` activity you just created will have the `partnerLink` attribute automatically filled in.

CONFIGURE THE <INVOKE> ACTIVITY

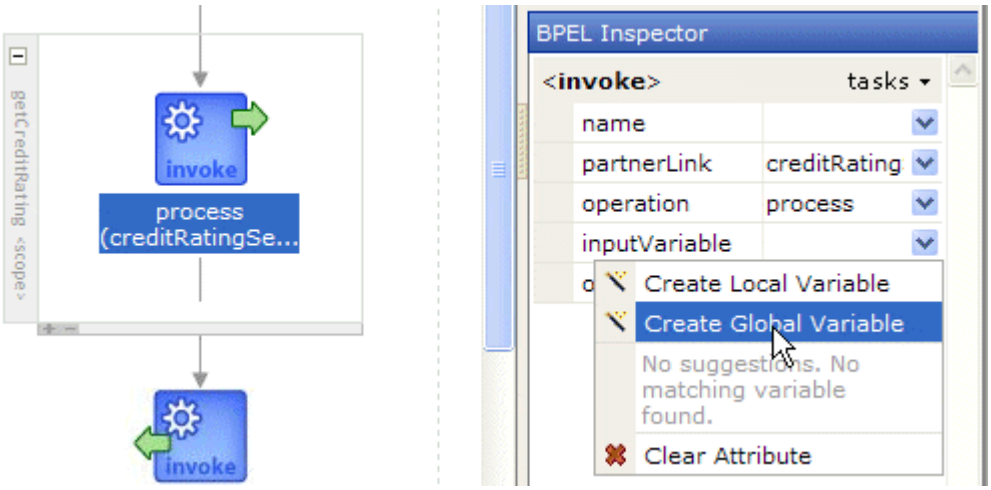
Now you will specify the operation you want to invoke for the service. Here the only operation defined by the service is called `process`.

1 In the BPEL Inspector, select `process` in the `operation` list.

After taking this step, you will use the drop-down arrows to the right of the `inputVariable` and `outputVariable` fields. The commands in these drop-down lists will bring up wizards for creating new variables, because the BPEL Designer knows from the WSDL file that the `process` operation has both input and output variables (and what their types are).

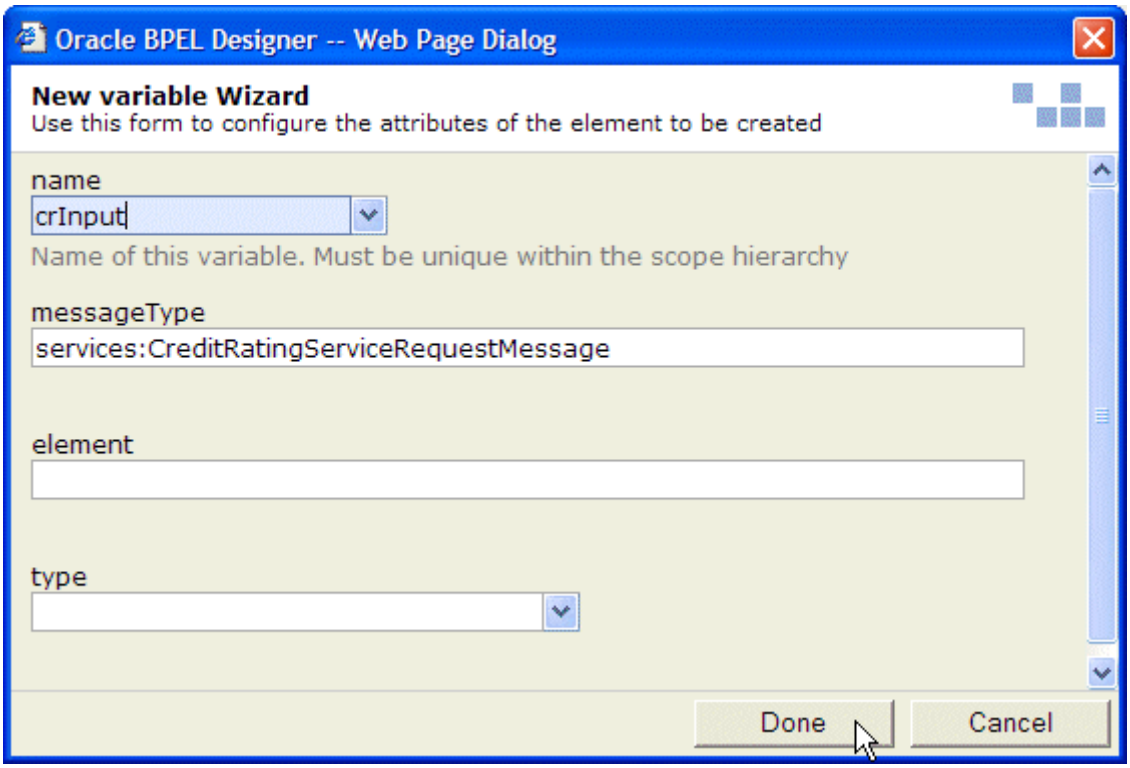
Note: The Inspector doesn't show `portType` as one of the `<invoke>` activity's attributes because that attribute is fully specified by the partnerLink. If you look in the BPEL source code, however, you will see that `portType` is set automatically.

2 In the `inputVariable` list, select **Create Global Variable**.



This will bring up the New variable Wizard, with the `messageType` field already filled in with the appropriate type. In the next step, you will enter the variable name. The remaining two fields enable you to specify XML schema elements or built-in XML schema simple types.

3 Enter a variable name of `crInput` and click **Done**.



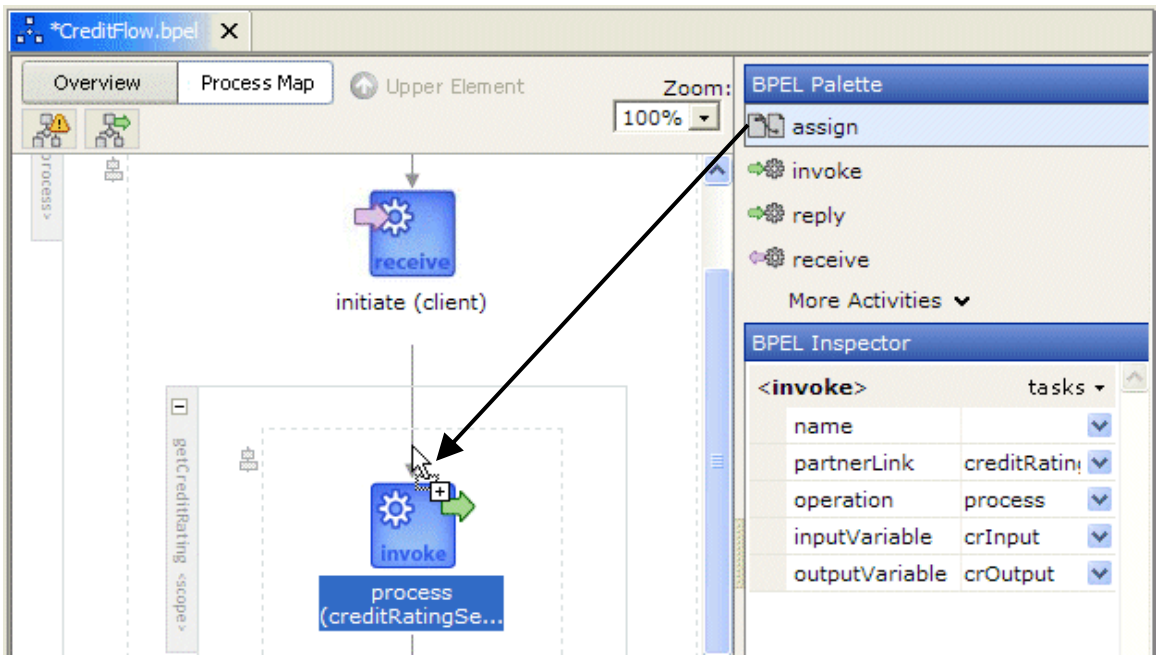
Note that you should only fill in one of the three type fields: `messageType`, `element`, or `type`. The next release of the BPEL Designer will enforce this by requiring you to select the type field you will use with a radio button.

4 Do the same for the `outputVariable` attribute to create a global variable named `crOutput`, accepting the automatically specified messageType.

INITIALIZE THE INPUT VARIABLE

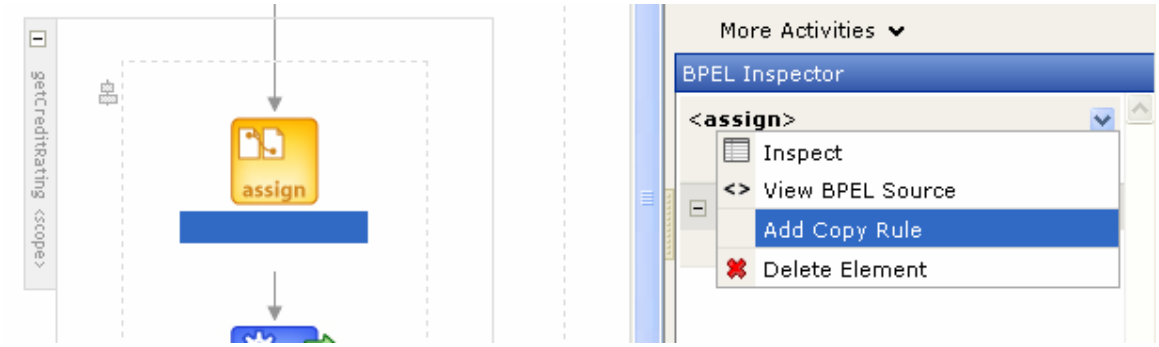
Now you will use XPath and the BPEL `<assign>` activity to perform simple data manipulation to initialize the input variable you are passing to the credit rating service.

1 Drag an `<assign>` activity from the BPEL Palette into your flow just before the invocation of the credit rating service (but within the `getCreditRating` scope).



As before, the newly created `<assign>` activity is now selected, and you can use the BPEL Inspector to configure its attributes.

2 In the Inspector, click the drop-down arrow in the upper-right of the `<assign>` section (called the **tasks drop-down)** and click **Add Copy Rule**.



Now you will use the Copy Rule form to copy the `ssn` field from your flow's input message into the `ssn` field of the credit rating service's input message.

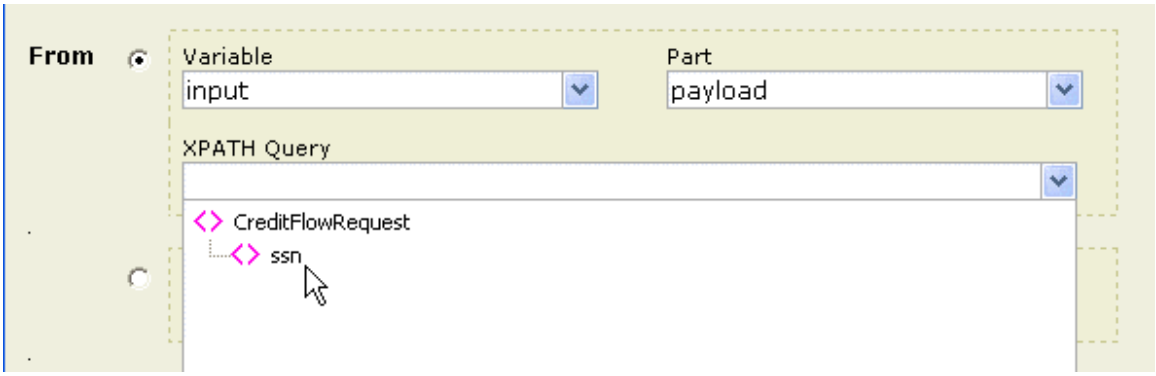
3 Fill in the **From** section of the Copy Rule form as follows:

a. In the **Variable** list, select `input` (which is the variable passed as input to kick off your BPEL process).

Once you select a variable, the **Part** list is populated with the appropriate values, based on the BPEL Designer's reading of the XML Schema type definition for this variable type.

b. In the **Part** list, select `payload`.

c. Click the **XPATH Query** field's drop-down arrow and, in the tree view displayed by the XPath query editor, select `ssn`. This enters the query `/tns:CreditFlowRequest/tns:ssn` in the field.



4 If you want to understand where the XPath query above comes from, take the following steps to view related parts of the source code (or simply read along and look at the extracted code below).

a. Click **Done** to close the Copy Rule form in its half-finished state.

b. Click the **BPEL Source** tab to view the `CreditFlow.bpel` source code, and notice that the `input` variable is of type `CreditFlowRequestMessage`.

```
<!-- Reference to the message passed as input during initiation -->
<variable name="input" messageType="tns:CreditFlowRequestMessage"/>
```

c. In the Navigator, drag `CreditFlow.wsdl` into the editor pane area and notice that the `CreditFlowRequestMessage` message type is defined as follows:

```
<message name="CreditFlowRequestMessage">
        <part name="payload" element="tns:CreditFlowRequest"/>
</message>
```

From this you can see that the part named `payload` will return an XML element of type `CreditFlowRequest`, where `CreditFlowRequest` is defined in the WSDL file as:

```
<element name="CreditFlowRequest">
       <complexType>
           <sequence>
               <element name="ssn" type="string"/>
           </sequence>
       </complexType>
    </element>
```

From the definitions above, you can see that the XPath query to get from the part named `payload` to the `ssn` field is `/tns:CreditFlowRequest/tns:ssn`.

d. Return to the `CreditFlow.bpel` window, click the **BPEL Designer** tab, and bring back the Copy Rule form by clicking **copy** under **Copy Rules** in the Inspector.

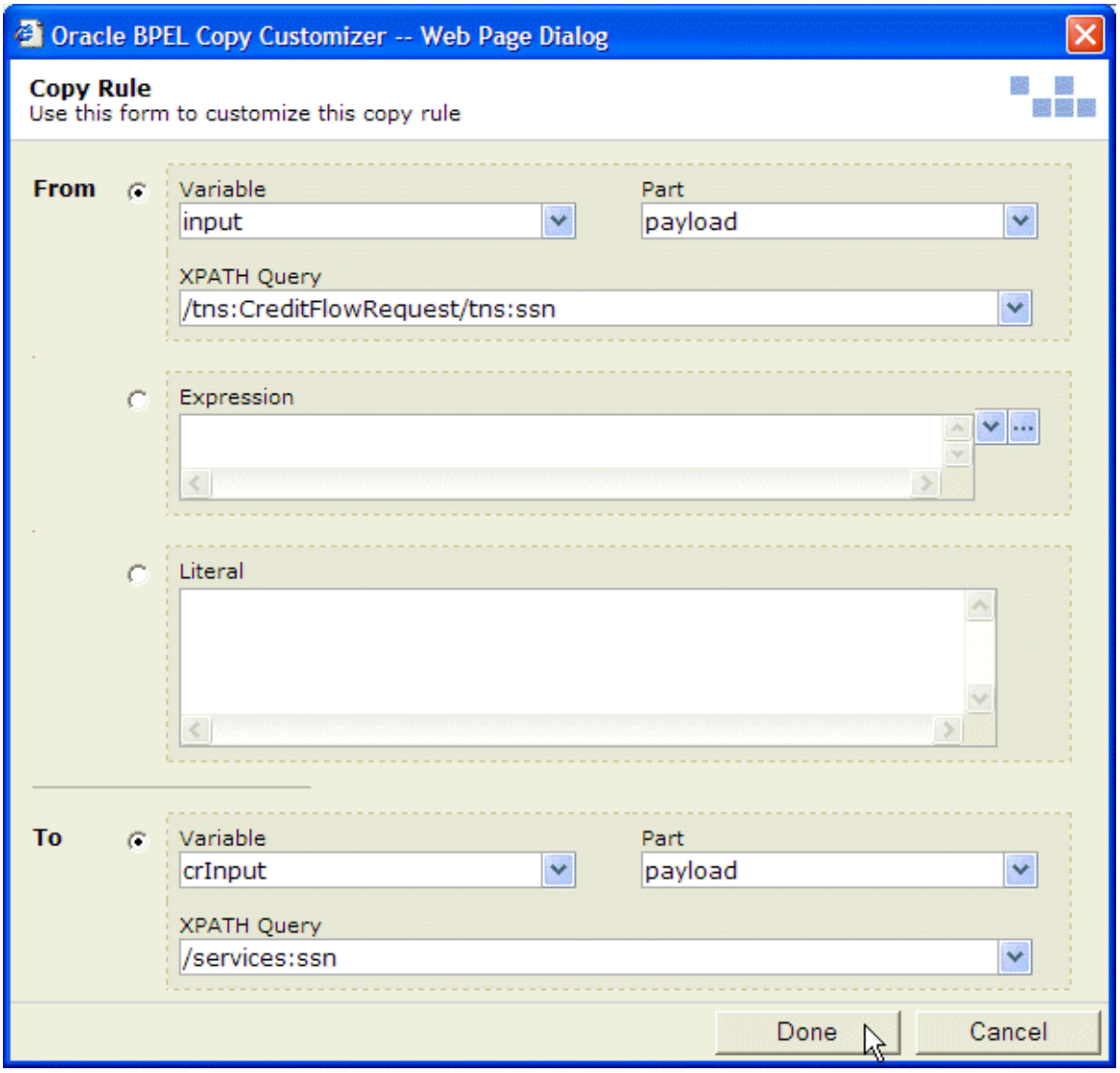
5 Fill in the **To** section of the Copy Rule form as follows:

a. In the **Variable** list, select `crInput`.

b. In the **Part** list, select `payload`.

c. In the XPath query editor, select `ssn` (which will enter the query `/services:ssn` in the XPath query field).

6 In the Copy Rule form, click **Done**.



Note that you can always enter XPath queries directly into the text field if you know what the correct query is. For more information regarding data manipulation in BPEL please refer to the BPEL Data Manipulation tutorial at http://otn.oracle.com/bpel, C:\orabpel\samples\tutorials\114.XSLTTransformations and C:\orabpel\samples\tutorials\115.XQueryTransformations

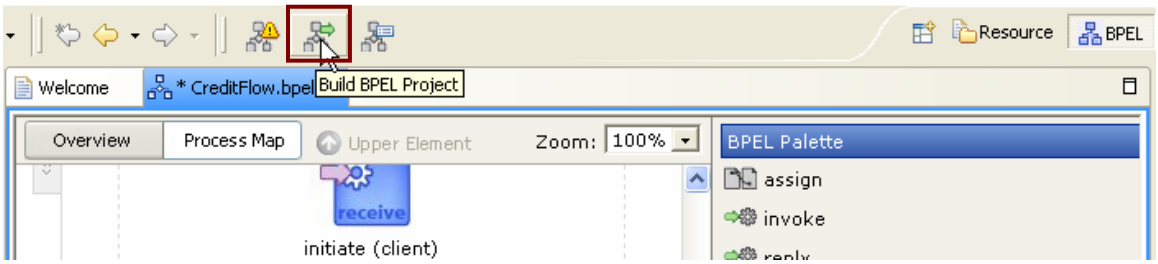
COMPILE, DEPLOY, AND TEST YOUR BPEL PROCESS

Although you have not yet wired up the return value from the credit rating service to the return value of your flow, you can still test your flow. In this section of the tutorial, you will compile, deploy, and test your BPEL process.

To compile and deploy your BPEL process:

1 Save the process.

2 Click the Build BPEL Project button, as shown below, to compile the process and deploy it to your local server's default domain.

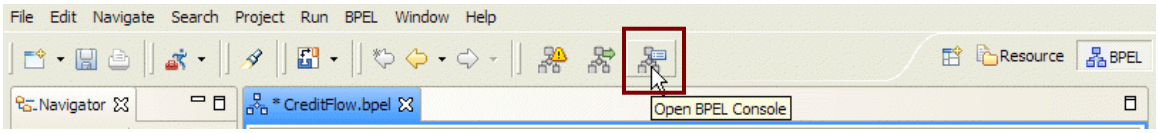


As the final step, you will test your BPEL process by using the automatically generated test interface in the BPEL Console (similar to what you did at the beginning of this tutorial, if you tested the deployed credit rating service).

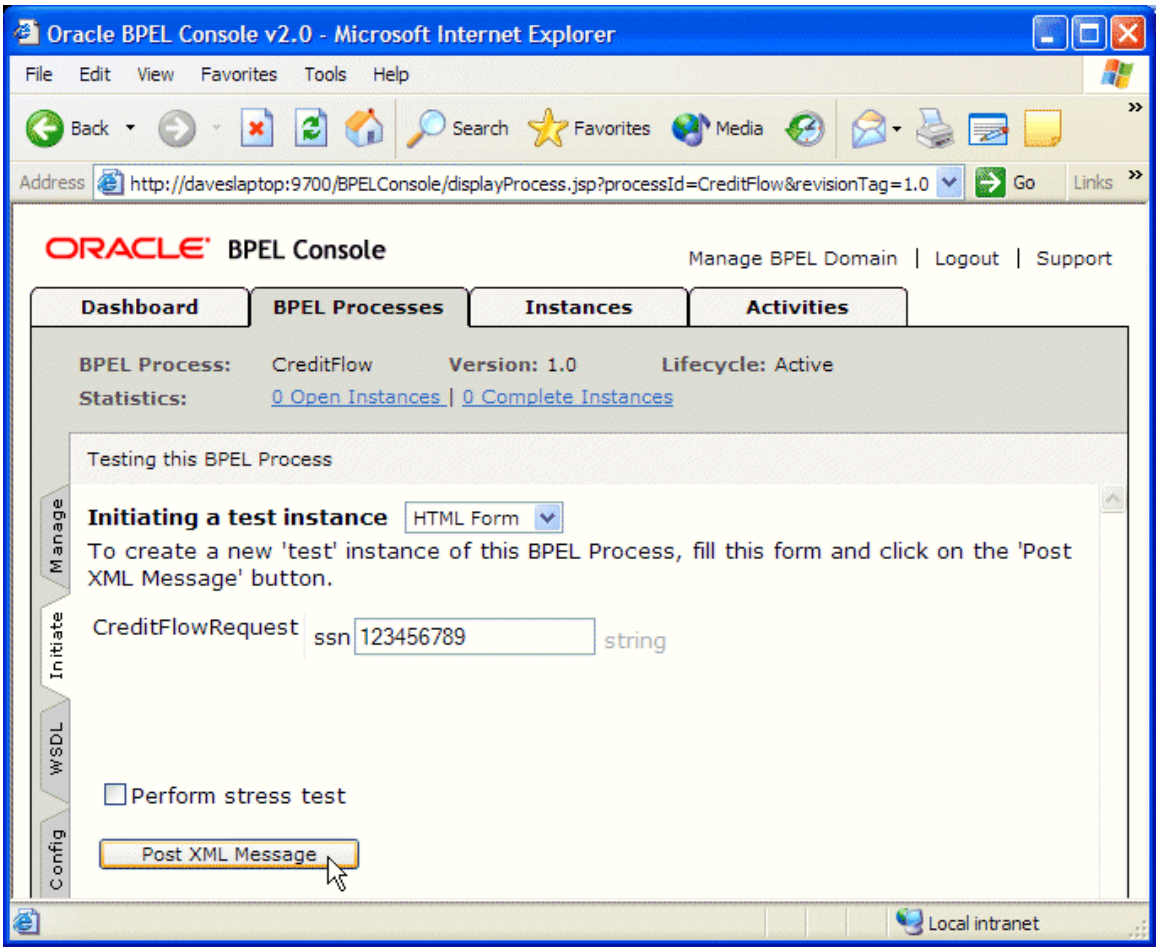
To initiate a test instance of your BPEL process:

3 Bring the BPEL Console into view.

Note that you can use the Open BPEL Console link in the Designer as a shortcut for bringing up the console when working in the Designer:

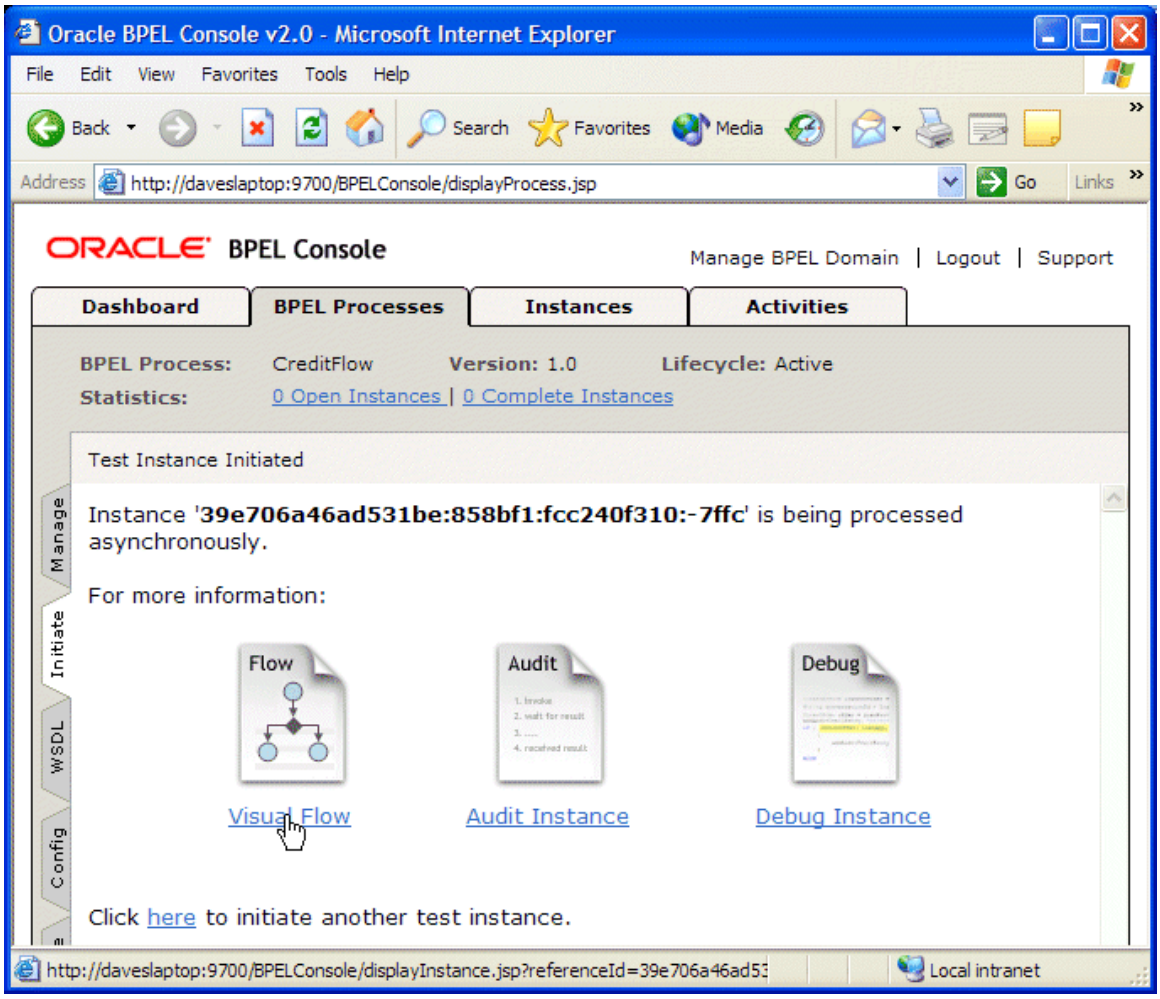


4 Click `CreditFlow` on the Dashboard and in the automatically generated HTML Form interface that appears, enter a nine-digit number that does *not* begin with a 0, and click **Post XML Message** to initiate the process.



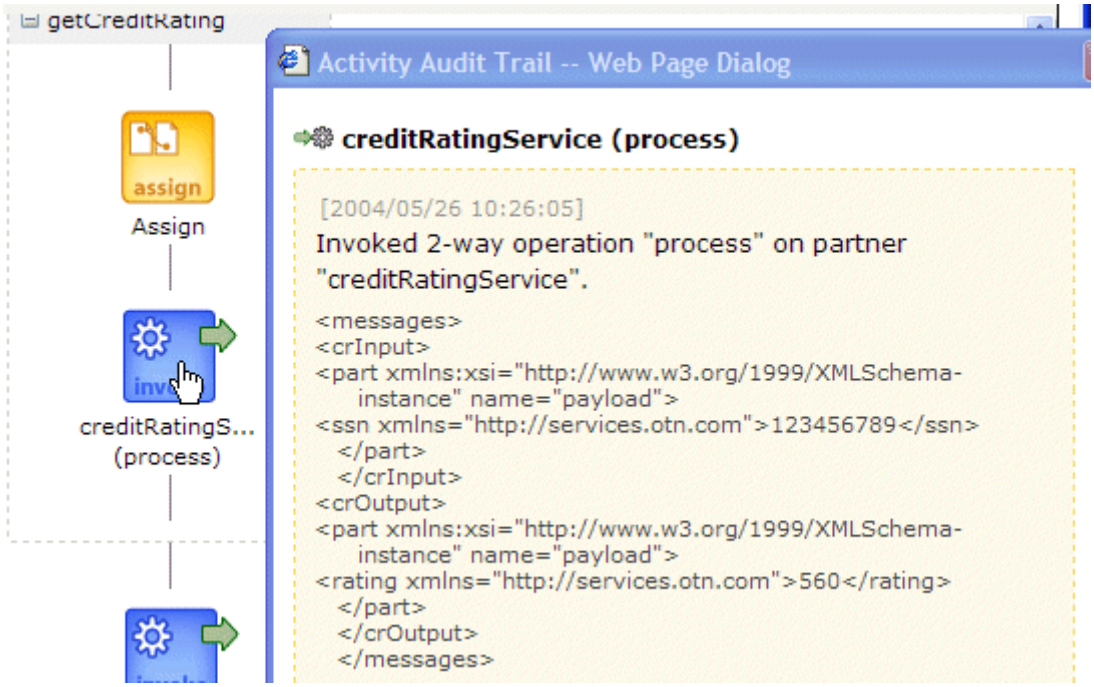
To view the visual audit trail of the instance:

5 Click the **Visual Flow** link to see a visual audit trail representing the current state of the process instance.



You will see an audit trail displaying the current state of the process, very similar to the process map displayed by the BPEL Designer. It will show that you have successfully invoked the credit rating service.

6 Click the `creditRatingService` `<invoke>` activity in the audit trail (a few boxes down) to see the messages sent to and received from the credit rating service.



To complete the implementation of this flow, you would add another `<assign>` activity to the flow (after the credit rating service has been invoked), which would copy the result returned from the credit rating service (in your `crOutput` variable) into the `creditRating` field for the result of your process itself (the `output` variable). We leave this as an exercise for you.

If you experience any difficulties in completing the flow or have any questions or comments regarding this tutorial, please submit your comments at http://otn.oracle.com/bpel.

Example II: A Loan Procurement BPEL Process

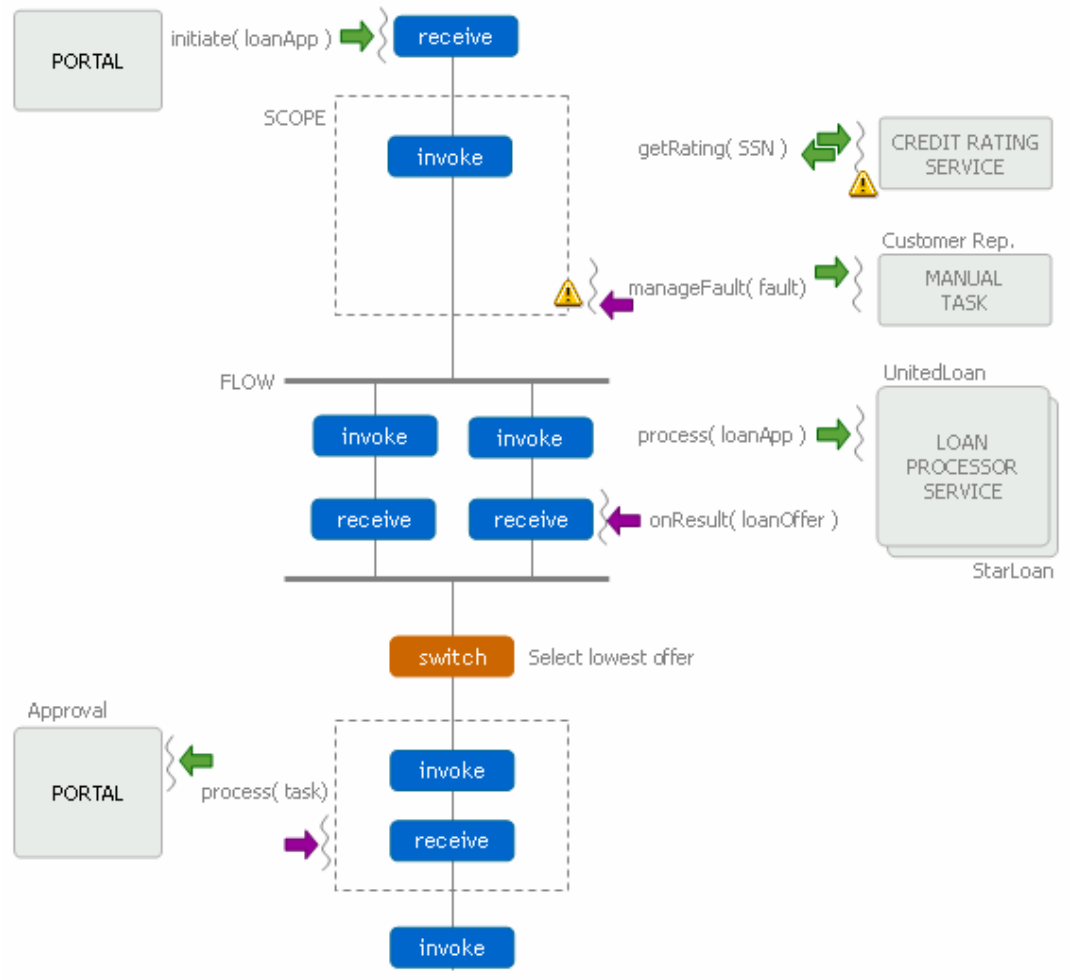
In this example we examine how a more sophisticated BPEL process can be implemented and executed using the Oracle BPEL Process Manager.

THE LOAN FLOW PLUS PROCESS

Most business processes will be much more complex than the simple example you created in the previous section. Fault-handling, interaction with asynchronous services, user/manual tasks, Java/J2EE code integration and custom user interfaces/portals are all key requirements for many business flows. Here you will see how you would build, test, debug and manage a BPEL process that implements all of these requirements.

The process is called “LoanFlowPlus” and is shipped with the samples for any Oracle BPEL Process Manager installation in the directory:
samples\demos\LoanDemoPlus\LoanFlowPlus

The requirements implemented by this application are shown in the diagram below:



This process takes a LoanApplication document as input and then will asynchronously return a selected and approved loan offer after an arbitrary amount of time. At the beginning of the execution of the process, it will fetch a social security number for the customer from an EJB and then go out to the synchronous CreditRating service that you saw previously to request a credit rating for the customer. However, in this case, the process is coded to handle the NegativeCredit exceptions (called “faults” in the SOAP/WSDL universe) which may be thrown by the credit rating service if a negative credit event is seen for the customer (for example a bankruptcy). If such an exception is thrown, the LoanFlowPlus process will queue up a task for a customer service rep to manually handle the exception and then wait for the task to complete. The customer service rep can use a dashboard UI to see their tasks and specify a credit rating or choose to cancel each application.

If a credit rating is supplied, either through manual or automated activities, the process will fall through into automated processing. It will then send the completed loan application to two loan processors, who can each take an arbitrary amount of time before returning loan offers. Since the loan processors can be long-running, they are implemented as asynchronous services and the BPEL process will invoke them in parallel.

In the case of this flow, it waits for both loan offers to be received before using some simple branching logic to select the best offer (the one with the lowest interest rate). Finally, another user task allows the customer to review and approve the selected loan offer. Once this task is completed, the selected and approved offer is returned as the result of the flow.

REVIEW THE PROCESS IN THE BPEL DESIGNER

Build and deploy external dependencies

As in the previous section, the LoanFlowPlus process is dependent on several external services which you can quickly build and deploy using the ‘obant’ command. To do this:

1 Open up a command prompt if you do not already have one open.

2 Change to the `orabpel\samples\demos\LoanDemoPlus` directory as follows:

```
> cd C:\orabpel\samples\demos\LoanDemoPlus
```

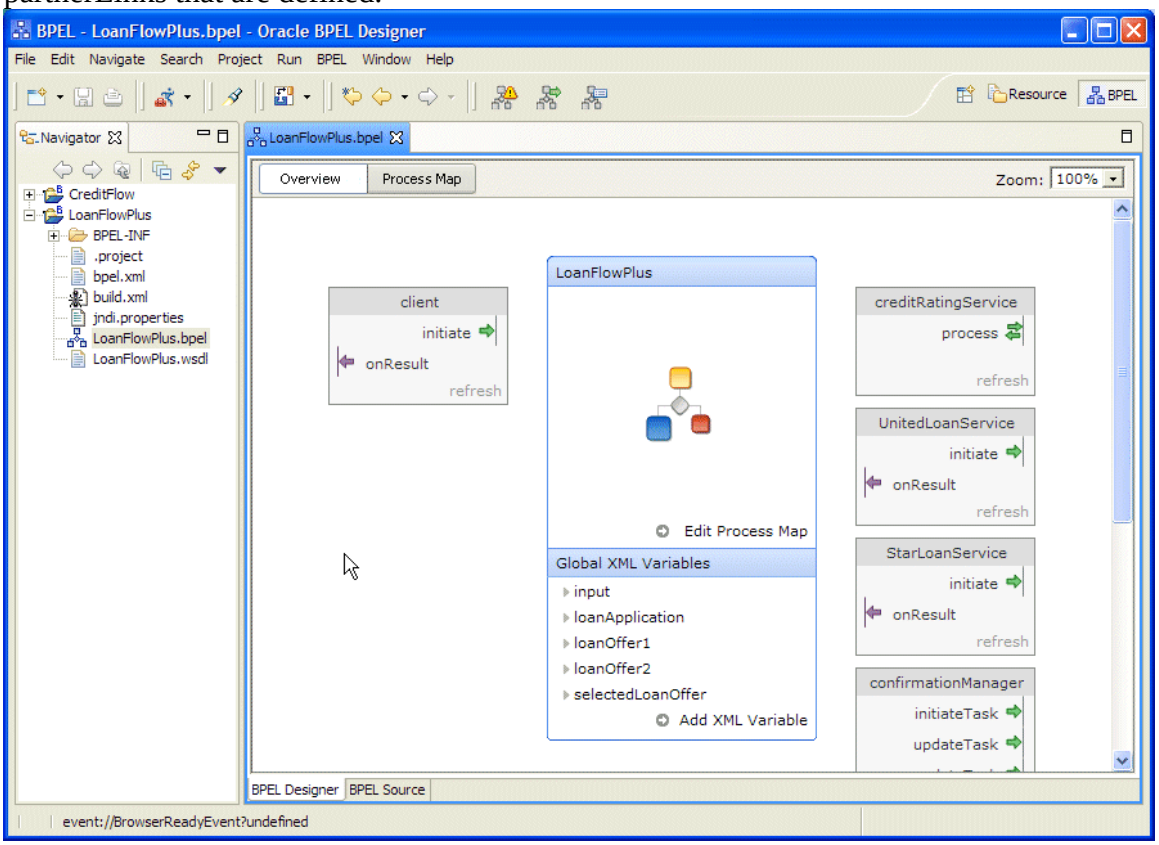
3 Execute the **obant** command.

```
> obant
```

Launch the BPEL Designer

4 You can now use the BPEL Designer to review the BPEL implementation of the LoanFlowPlus process. To do this, start up the Designer as you did in the previous section, and choose the File / Import menu command. Select “Existing Project into Workspace” and then browse to `C:\orabpel\samples\demos\LoanDemoPlus\LoanFlowPlus` and click Finish to import the LoanFlowPlus process into the Designer.

5 Double-click on the LoanFlowPlus.bpel file in the Navigator and you will see the overview in the Designer showing client interface to the process and the different partnerLinks that are defined.



You can see that the creditRatingService is the same synchronous service used in the previous section, however the StarLoan and UnitedLoan services are asynchronous, supporting a one-way initiate operation and an asynchronous onResult callback which will be called after a loan offer is ready from each service – which could take anywhere from a few seconds to a few days or more. As with your previous process, the client interface to the LoanFlowPlus process itself is also asynchronous.

6 If you select the Process Map view in the Designer, you will see a visual representation of the whole process. Keep in mind that the BPEL Designer uses 100% standard BPEL source as its native format so you can always select the BPEL Source view to see the underlying BPEL source code and/or make changes in either view and see those changes represented immediately in the alternate view. Here you will see that the BPEL Designer view shows that the process gets initiated, then retrieves the ssn, then encapsulates the logic to fetch the credit rating (including the exception handling code) in a BPEL <scope>, etc.

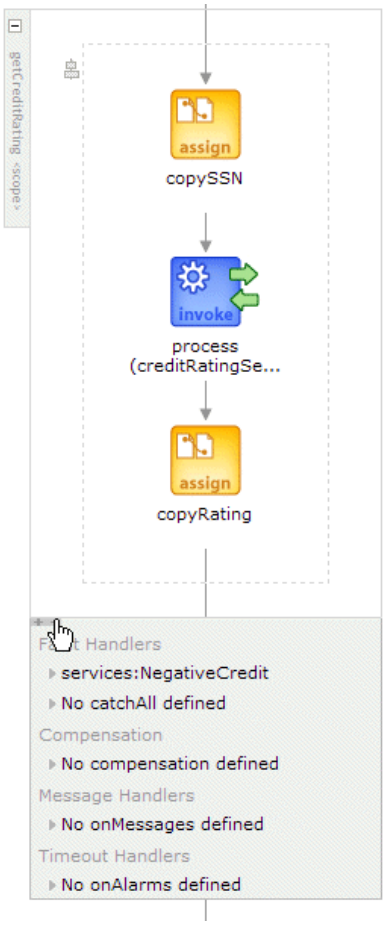
In the next few sections of this document, we will review the code which implements this process, including how it is built and represented in the Designer's graphical view and the underlying BPEL source.

MANAGE FAULTS

The LoanFlowPlus process uses the BPEL fault handling mechanism to catch and manage exceptions thrown by the credit rating service. Fault handlers are associated with a particular <scope> activity and faults encountered within a scope and not handled by that scope are bubbled up to the enclosing scope, just like try-catch in Java.

7 To see the NegativeCredit faultHandler defined for the getCreditRating scope, select the "+" to the left of the getCreditRating scope to open up the view of that scope.

8 Then click the "+ -" link at the bottom left of the expanded scope to see the faultHandler(s) defined for that scope.



You can click the link for a faultHandler, such as the "services:NegativeCredit" link and you will see the steps which are executed as the logic for handling this fault. In BPEL any activity (which can be arbitrarily complex, short or long-running, etc) can be used in a

faultHandler. In this case a scope is defined which creates a user task for a customer service rep to handle the exception in a manual fashion. Of course faults can also be handled via automated logic or re-thrown.

We won't look at the details of the BPEL source which implements this fault handler, but a skeleton of the code is shown below.

```
<scope name="getCreditRating" variableAccessSerializable="no">
    <variables>
        ...
    </variables>
    <!-- Watch for faults (exceptions) being thrown from
         creditRatingService -->
    <faultHandlers>
        <catch faultName="services:NegativeCredit"
               faultVariable="crError">
            <!-- Actual fault handling code goes here -->
            ...
        </catch>
    </faultHandlers>
    <sequence>
        <!-- Code during which you want to watch for faults -->
        ...
        <invoke name="invokeCR" partnerLink="creditRatingService"
                                portType="services:CreditRatingService"
                                operation="process"
                                inputVariable="crInput"
                                outputVariable="crOutput"/>
        ...
    </sequence>
</scope>
```

Note that if you did click on the fault to see the fault handling logic, the "Upper Element" link at the top of the main window will return you to the Process Map for the overall process.

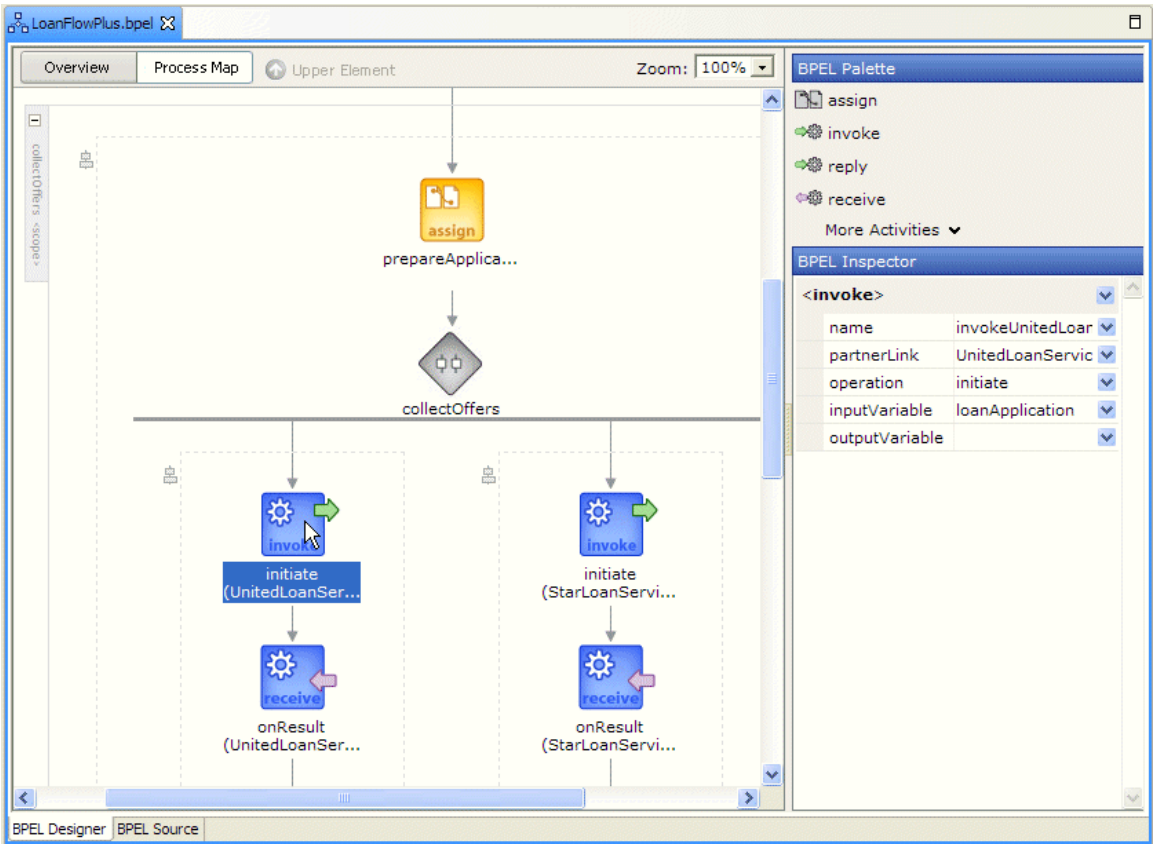
For more information about exception management, reliability and crash testing, please refer to the technotes for "Building resilient BPEL processes" and "Running crash tests on the Oracle BPEL Server" at http://otn.oracle.com/bpel.

INTERACT WITH AN ASYNCHRONOUS LOAN PROCESSOR SERVICE

Once the loan application document has been filled in, it will be sent to the two asynchronous loan processor services. In this section, we examine what is required to interact with an asynchronous service in a BPEL process. As mentioned previously, BPEL has support for asynchronous activities at its core and the Oracle BPEL Process Manager supports a "dehydration" capability so that flows are reliably and efficiently persisted to a datastore, along with all their current state information, whenever they are waiting for asynchronous events.

In addition, BPEL enables support for several standard methods of correlating asynchronous messages so that asynchronous callbacks can find their way back to the appropriate waiting process instance. Specifically, the current release of the Oracle BPEL Server supports both the correlation set mechanism, which uses message content for correlation, and the WS-Addressing specification, which uses SOAP message headers for correlation of asynchronous messages.

To see how the LoanFlowPlus process invokes the UnitedLoan service and waits for its asynchronous callback, expand the “collectOffers” scope and select the invoke activity in the left hand sequence labeled “initiate (UnitedLoanService)”, as shown below.



In the BPEL Inspector pane, you can see that this activity passes the loan application document as an input variable to the initiate operation for the UnitedLoan service. This initiate operation will return immediately, but the next activity, the <receive> for the onResult callback, will wait until the service has called back with a loan offer. WS-Addressing is used for message correlation (by default) and is handled completely transparently by the BPEL Server. (Note that if you want to see the WS-Addressing correlation information, you can use a TCP tunnel to see the SOAP messages exchanged with the services, per the tech note on “TCP Tunneling the Oracle BPEL Server” at http://otn.oracle.com/bpel.

The BPEL source code for the invoke and receive activities which get the offer back from UnitedLoan can be seen by using the BPEL Source view or by right-clicking on the sequence icon for the UnitedLoan sequence and selecting “View BPEL Source”. This source code is shown below:

```
<sequence>
    <!--  initiate the remote service -->
    <invoke name="invokeUnitedLoan"
            partnerLink="UnitedLoanService"
            portType="services:LoanService"
            operation="initiate"
            inputVariable="loanApplication"/>
```

```
    <!--  receive the result of the remote service -->
    <receive partnerLink="UnitedLoanService"
             portType="services:LoanServiceCallback"
             operation="onResult"
             variable="loanOffer1"/>
</sequence>
```

PARALLEL PROCESSING

As mentioned in the requirements section, LoanFlowPlus needs to invoke the two loan service providers in parallel since they can take an arbitrary amount of time to complete. This can be done in BPEL using the <flow> activity, which allows several actions to be taken in parallel. An example of this can be seen in the LoanFlowPlus collectOffers flow shown above which contains two parallel activities – the sequence which invokes the UnitedLoan service and the sequence which invokes StarLoan.

The BPEL source which implements this flow activity is fairly simple and is shown below:

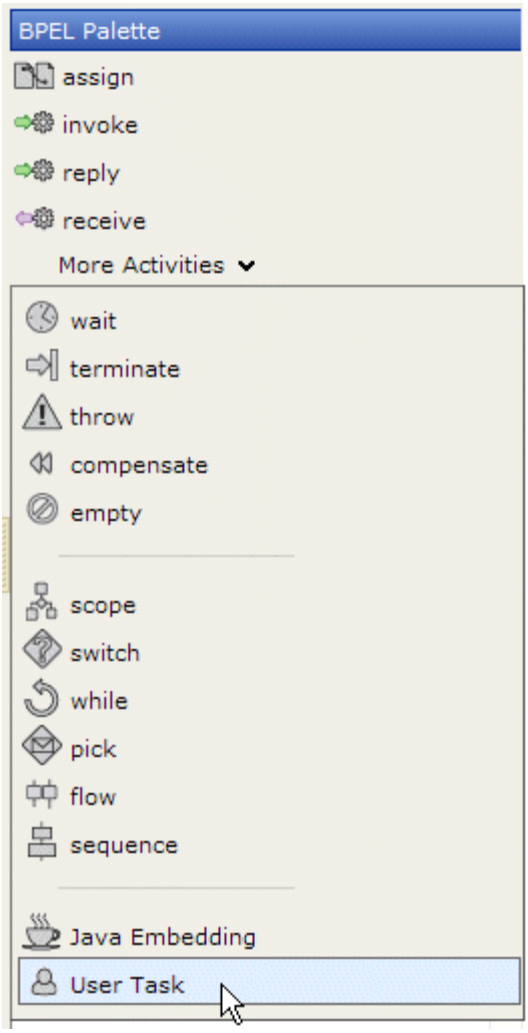
```
<flow name="collectOffers">
  <!-- invoke first loan provider -->
  <sequence>
     <!--  initiate the remote service -->
     <invoke partnerLink="UnitedLoanService" .../>

     <!--  receive the result of the remote service -->
     <receive partnerLink="UnitedLoanService" .../>
  </sequence>

  <!-- invoke the second loan provider (in parallel with the first) -->
  <sequence>
     <!--  initiate the remote service -->
     <invoke partnerLink="StarLoanService" .../>

     <!--  receive the result of the remote service -->
     <receive partnerLink="StarLoanService" .../>
  </sequence>
</flow>
```

USER TASKS

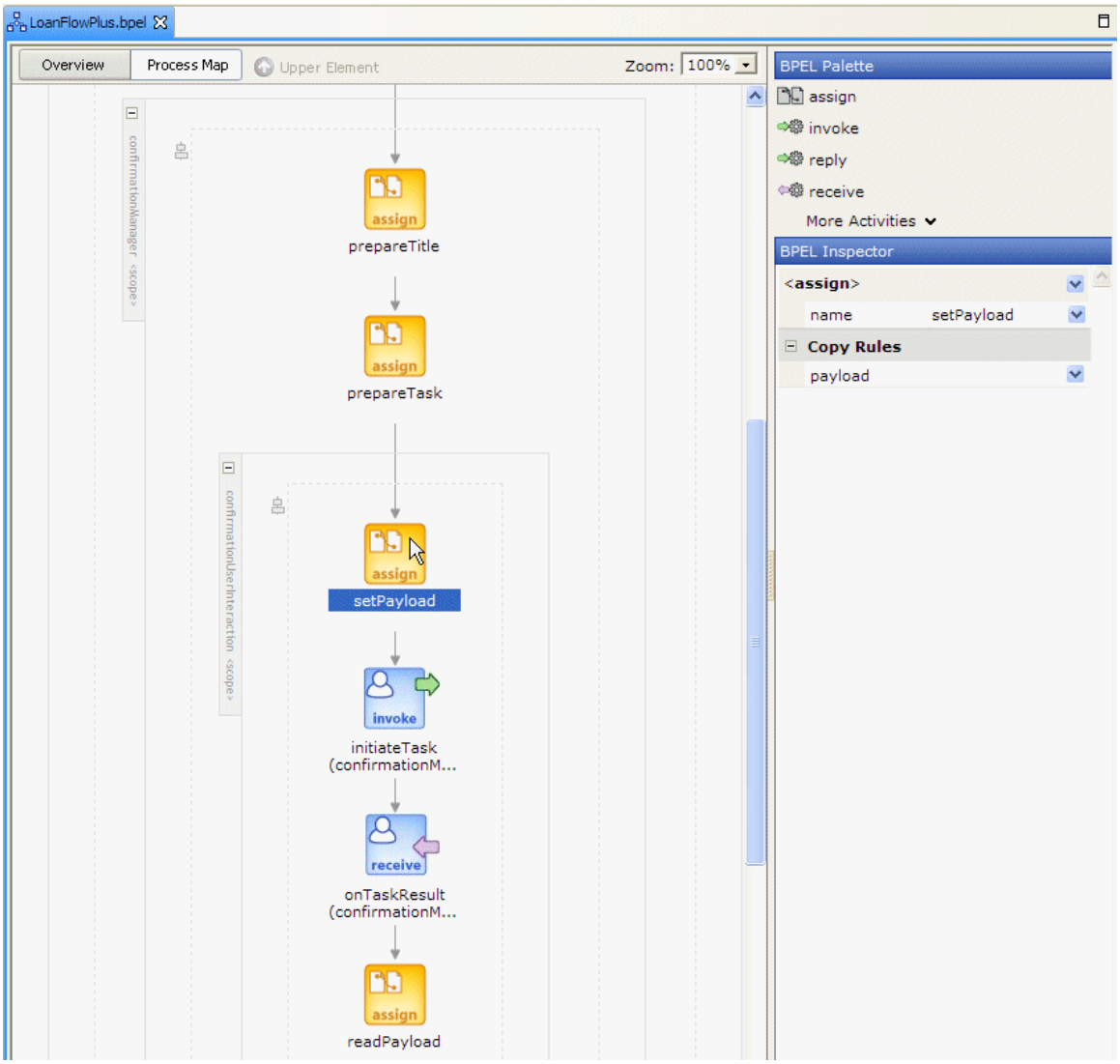
BPEL has fundamental support for asynchronous services which is easily used as a platform to integrate people and manual tasks into BPEL processes. Working with some key early adopter customers to define the functional requirements, Oracle bundles a manual task Web service with our BPEL Server. By implementing this as a true BPEL service, the interface to the task service is described with WSDL and people can be included in 100% standard BPEL processes - to the BPEL process, the person/manual task looks like any other asynchronous Web service.

The LoanFlowPlus application illustrates manual task support in several places: for exception management, as described previously, and for the approval step where the flow waits for the customer to confirm the selected loan offer before completing.

To see how the user confirmation is implemented, expand the confirmationManager scope in the BPEL Designer. Note that this entire scope, with the setup of variables and including the sub-scope named “confirmationUserInteraction” can be created in one step using the User Task activity that is included on the BPEL Palette in the Designer:



The User Task palette element is a templatized activity that allows the developer to treat a user task as a higher level abstraction, although it is implemented as a series of BPEL activities. As you can see if you expand both the “confirmationManager” scope and the “confirmationUserTask” sub-scope within this user task, adding a task to a BPEL process involves first setting up the input variable to be passed to initiate a new instance of the task service. This is done with 3 separate assign activities for this task: prepareTitle, prepareTask and setPayload.



The structure of the task data is defined by the TaskManagerService (and can be found for your local BPEL Server deployment at http://daveslaptop:9700/orabpel/default/TaskManager/Task.xsd). Specifically, it includes fields like the task title, assignee, expiration date as well as an attachment which allows arbitrary data to be passed to the task.

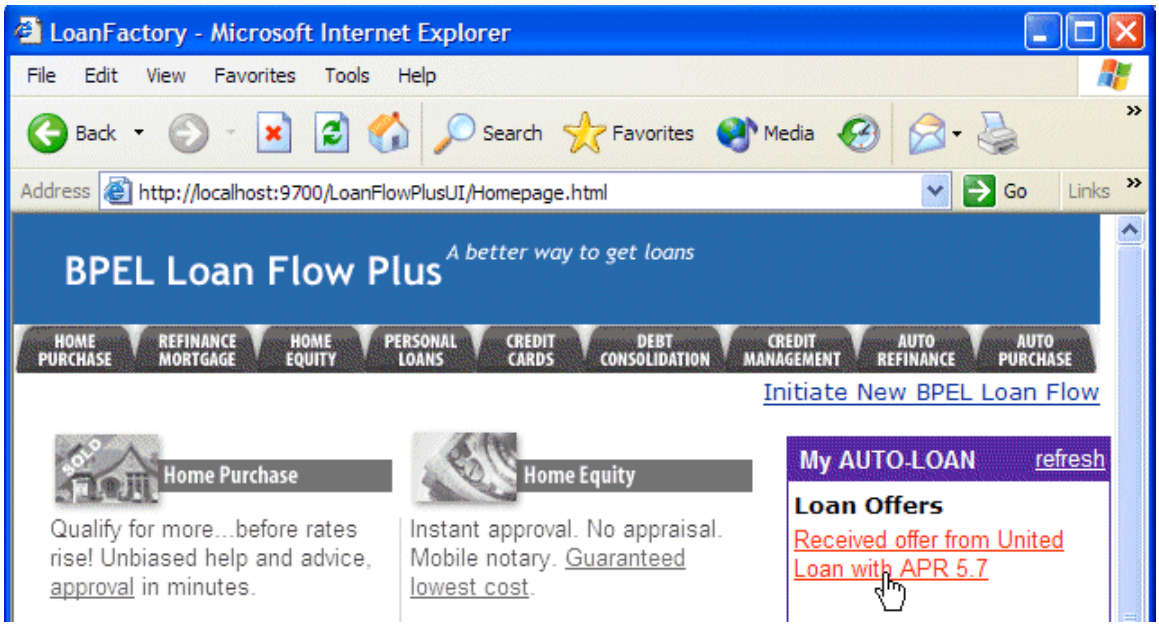
Once the task data is set up, the BPEL process initiates the task service and then waits for the task to complete, getting back updated task data upon completion. The pertinent section of the Process Map view for the confirmation user task in the LoanFlowPlus process is shown above.

The BPEL source code for this task service invocation looks like:

```
<scope name="confirmationManager" variableAccessSerializable="no">
    <variables>
        <variable name="taskTitle" type="xsd:string"/>
        <variable name="confirmationTask" element="task:task"/>
    </variables>
```

```
    <sequence>
        <assign name="prepareTitle">
            <!-- Set up title variable for task -->
            ...

        </assign>
        <assign name="prepareTask">
            <!-- Set up task data -->
            ...

            <!-- Assign 'attachment' to task request -->
            <copy>
                <from variable="selectedLoanOffer"
                      part="payload"
                      query="/loanOffer"/>
                <to   variable="confirmationTask"
                      query="/task/attachment"/>
            </copy>
        </assign>
        <scope name="confirmationUserInteraction"
               variableAccessSerializable="no">
            <variables>
                <variable name="taskRequest"
                          messageType="task:taskMessage"/>
                <variable name="taskResponse"
                          messageType="task:taskMessage"/>
            </variables>
            <sequence>
                <!-- Assign task document to task message -->
                <assign name="setPayload">
                    <copy>
                        <from variable="confirmationTask"/>
                        <to variable="taskRequest" part="payload"/>
                    </copy>
                </assign>
                <!--  initiate task -->
                <invoke name="initiateTask"
                        partnerLink="confirmationManager"
                        portType="task:TaskManager"
                        operation="initiateTask"
                        inputVariable="taskRequest"/>

                <!--  Receive the outcome of the task -->
                <receive name="receiveTaskResult"
                         partnerLink="confirmationManager"
                         portType="task:TaskManagerCallback"
                         operation="onTaskResult"
                         variable="taskResponse"/>

                <!-- Read task document from task message -->
                <assign name="readPayload">
                    <copy>
                        <from variable="taskResponse" part="payload"/>
                        <to variable="confirmationTask"/>
                    </copy>
                </assign>
            </sequence>
        </scope>
    </sequence>
</scope>
```

APIs are provided (both Web services/SOAP and Java) so that clients can query the tasks assigned to a user, update the task data and complete or cancel the task. In addition expiration of tasks, escalation and other common user task requirements are implemented by the task service itself. For the LoanFlowPlus process, a portal is used to display the loan offer(s) received by a user and allow an offer to be selected and approved.



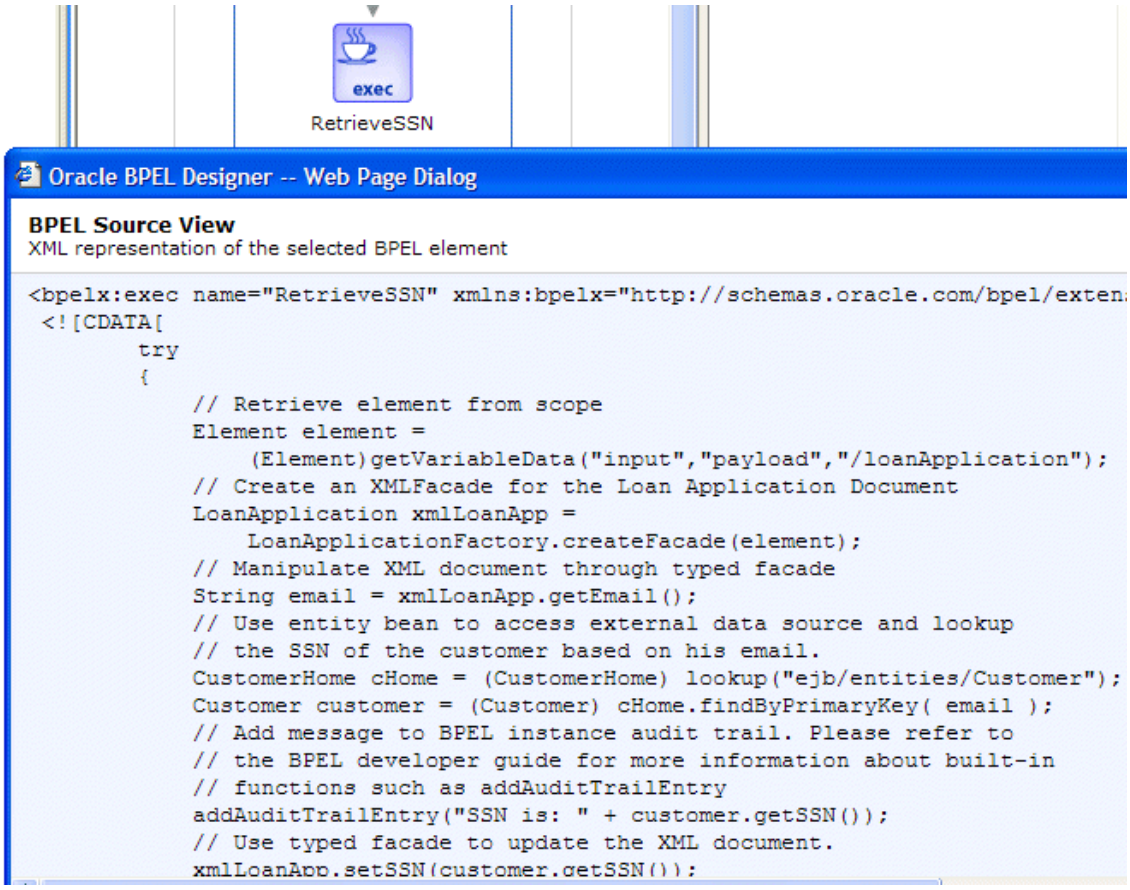
In this case, the Java API is used a JSP client which interfaces to the task service. For more information on the BPEL Task Service and building task service clients, see the TaskManager tutorial available at http://otn.oracle.com/bpel.

INLINE JAVA CODE SNIPPET

Frequently an enterprise has existing Java code or Java APIs for accessing back-end systems which it wants to leverage in a BPEL process. There are 3 general approaches for this: wrapping the Java code as a Web service; using the WSIF invocation framework to call the Java code as if it were a Web service; and “inlining” the Java code into a BPEL process. The first two approaches are fairly straightforward and illustrated by several code examples shipped with the Oracle BPEL Process Manager.

As an example of the third approach, the LoanFlowPlus process uses an EJB to fetch a social security number for the customer. In this case an entity bean is used, however code examples are available to illustrate how to inline stateless session beans and plain old Java code into BPEL processes. To support inlining Java code, the Oracle BPEL Process Manager uses the BPEL extension capability. This extension is generic enough to allow for any language code (e.g. C#) to be integrated and is being standardized as part of the JSR-207/BPELJ initiative.

To see how this is done in LoanFlowPlus, right-click on the Java exec node labeled “RetrieveSSN” (the first activity in the process) and select the View BPEL Source option. This will show the BPEL source which implements the selected activity, which in this case is the inlined Java code:



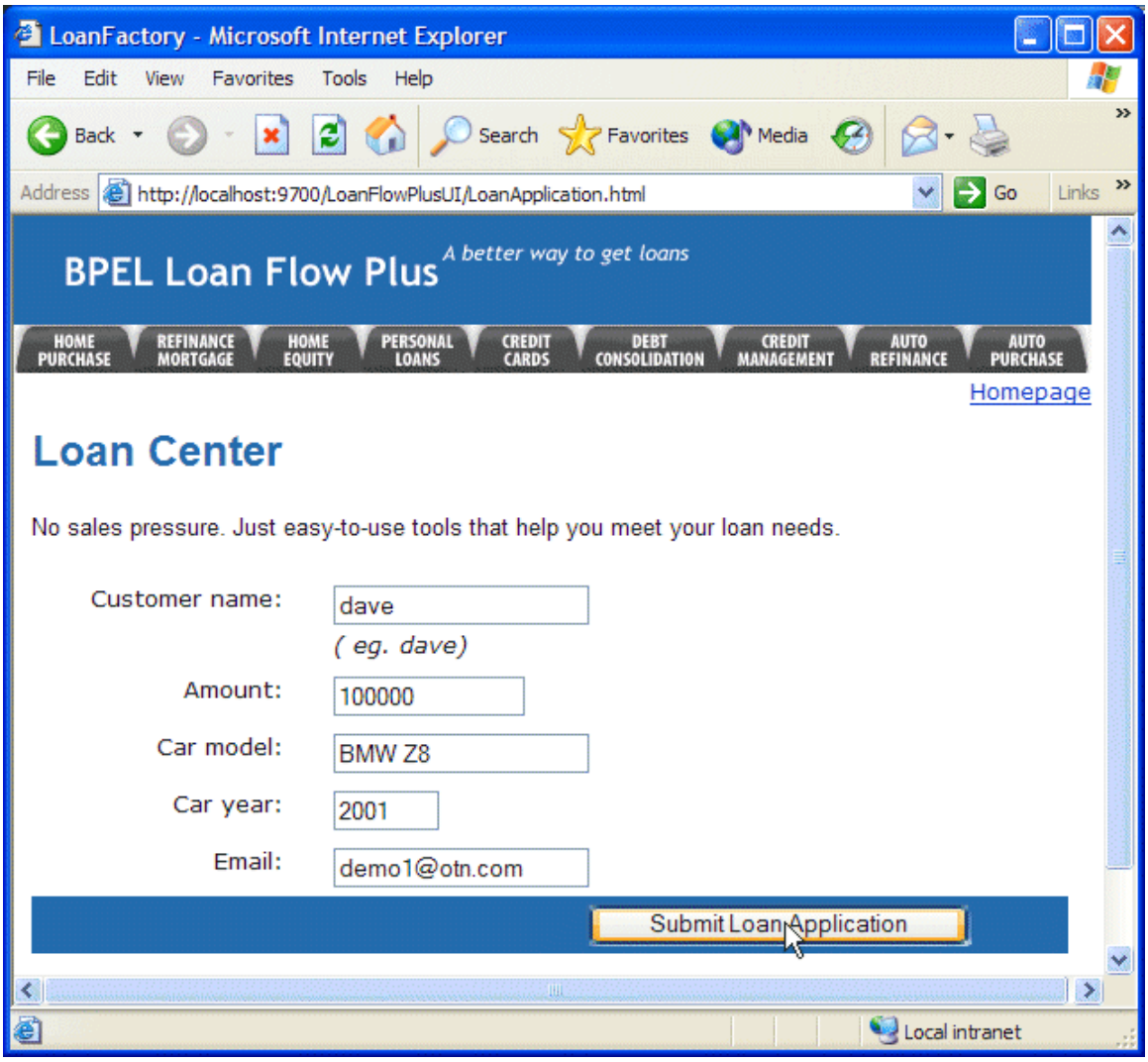
As you can see, this code is just standard J2EE code to invoke an EJB, plus a few utility APIs to access BPEL variables from Java code. To make this functionality easy to access, a templatized activity, called “Java Embedding” is available on the BPEL Palette. Dragging this activity onto a BPEL process will insert a template for using the Java exec activity in a BPEL process.

BUILD A CUSTOM USER INTERFACE FOR INITIATING THE BPEL PROCESS

All BPEL processes are themselves Web services. In addition, the Oracle BPEL Process Manager also provides a Java API for invoking deployed BPEL flows, fetching state/status information from active instances, etc. Frequently a BPEL process will be instantiated from a portal or other custom user interface – for example, the LoanFlowPlus process has a JSP loan portal that customers can use to initiate new loan applications, see the offers they have received and approve offers. The LoanFlowPlus UI is deployed onto the same application server as the Oracle BPEL Process Manager and full source for it can be found in your samples at:

C:\orabpel\samples\demos\LoanDemoPlus\LoanFlowPlusUI

As you will see when you run this application, the portal page which initiates new LoanFlowPlus instances looks like the following (but could be any user interface that is desired):



JavaDocs for the Java API for initiating a deployed BPEL process can be found in:

```
C:\orabpel\docs\apidocs\index.html
```

In addition, a JSP tag library is available to make it easy to use this Java API from a JSP (this tag library is used by the LoanFlowPlusUI sample) and developers can also use the Web services SOAP/WSDL API to invoke BPEL processes. The Web services approach allows BPEL flows to be invoked and accessed from any language or toolkit that supports Web services.

INITIATE THE LOANFLOWPLUS PROCESS

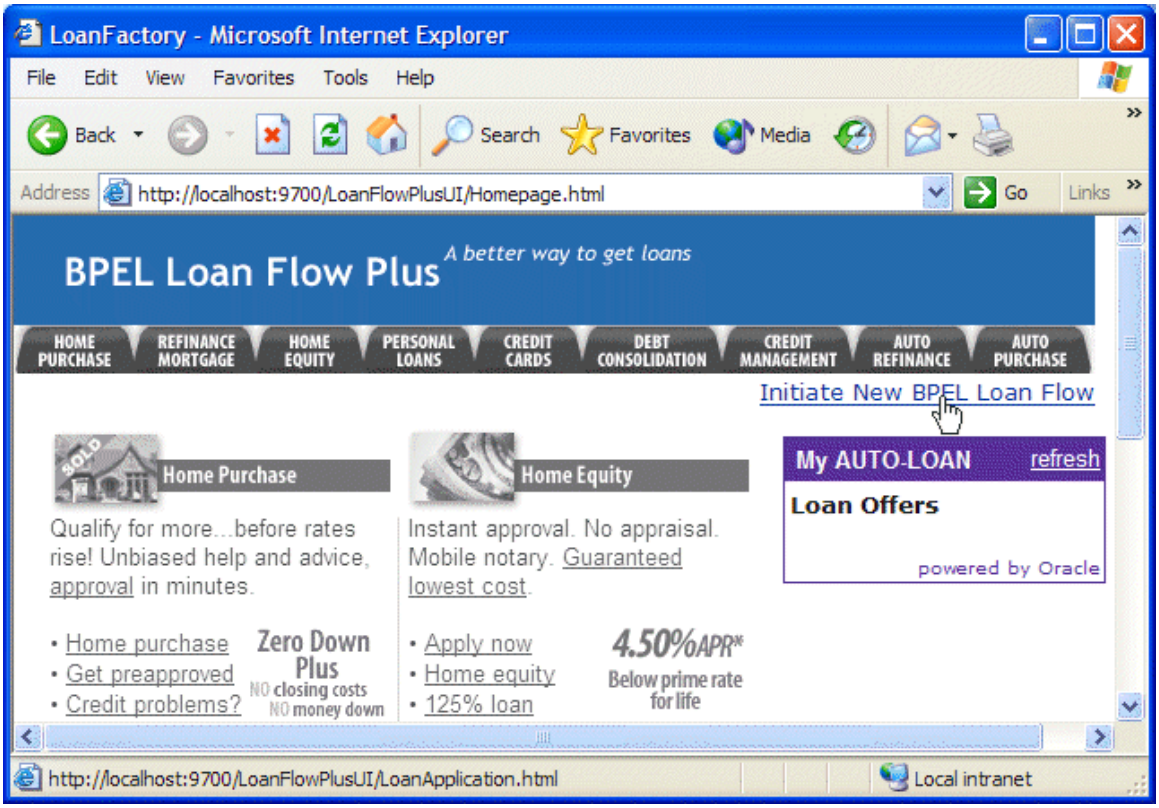
In this section you will use the portal UI to initiate and complete a LoanFlowPlus instance and the BPEL Console to view the status, audit trail, debugging information, performance metrics and other information gathered automatically by the Oracle BPEL Server.

9 The LoanFlowPlus BPEL process was compiled and deployed, along with its dependent services and the portal UIs, when you executed the obant command at the

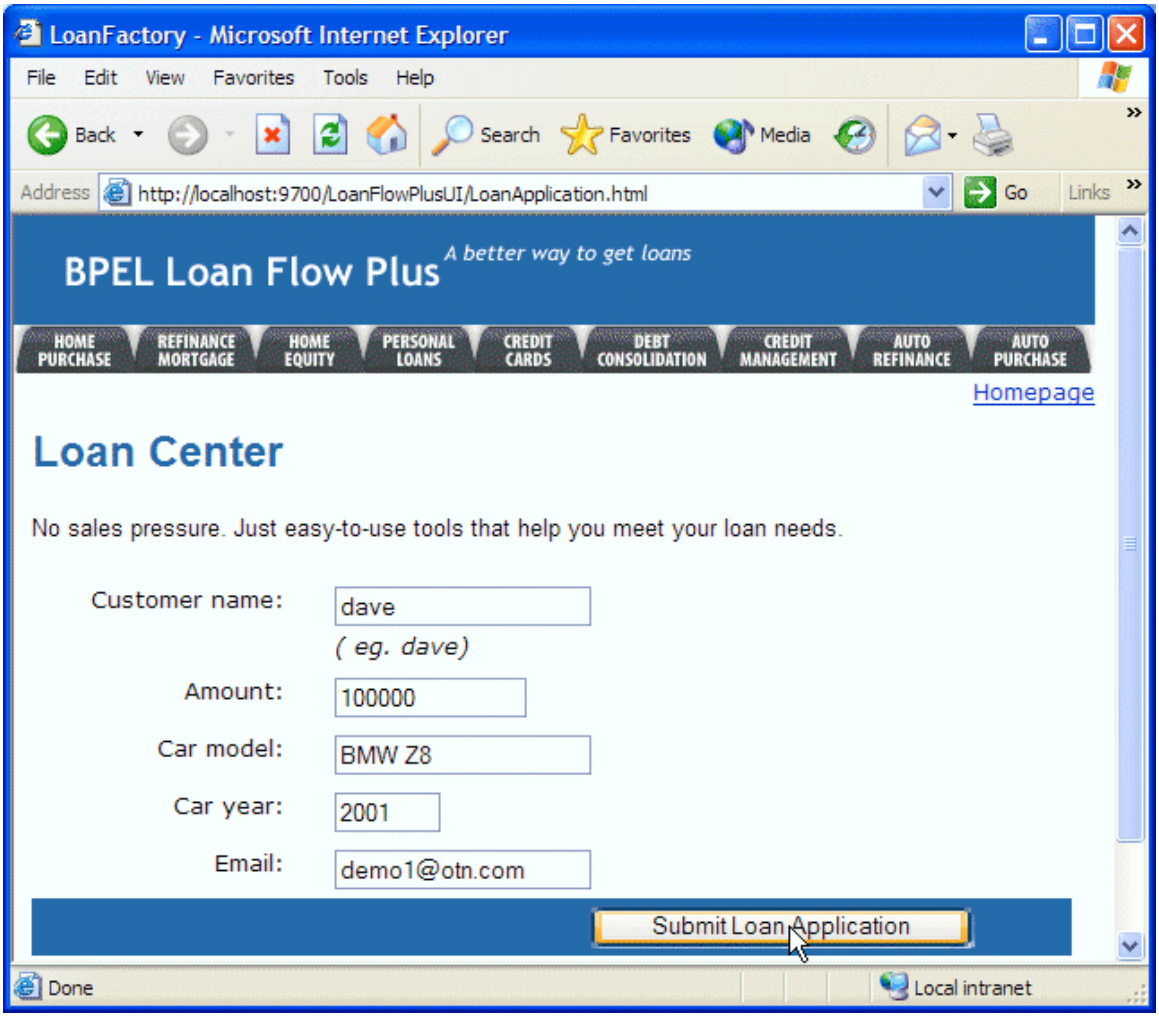
beginning of this section. You could of course also build and deploy it from the Designer, as you did in the previous section. You should now point a browser at the portal UI located at:

http://localhost:9700/LoanFlowPlusUI/Homepage.html

10 Select the "Initiate New BPEL Loan Flow" link on the portal page, as shown below:



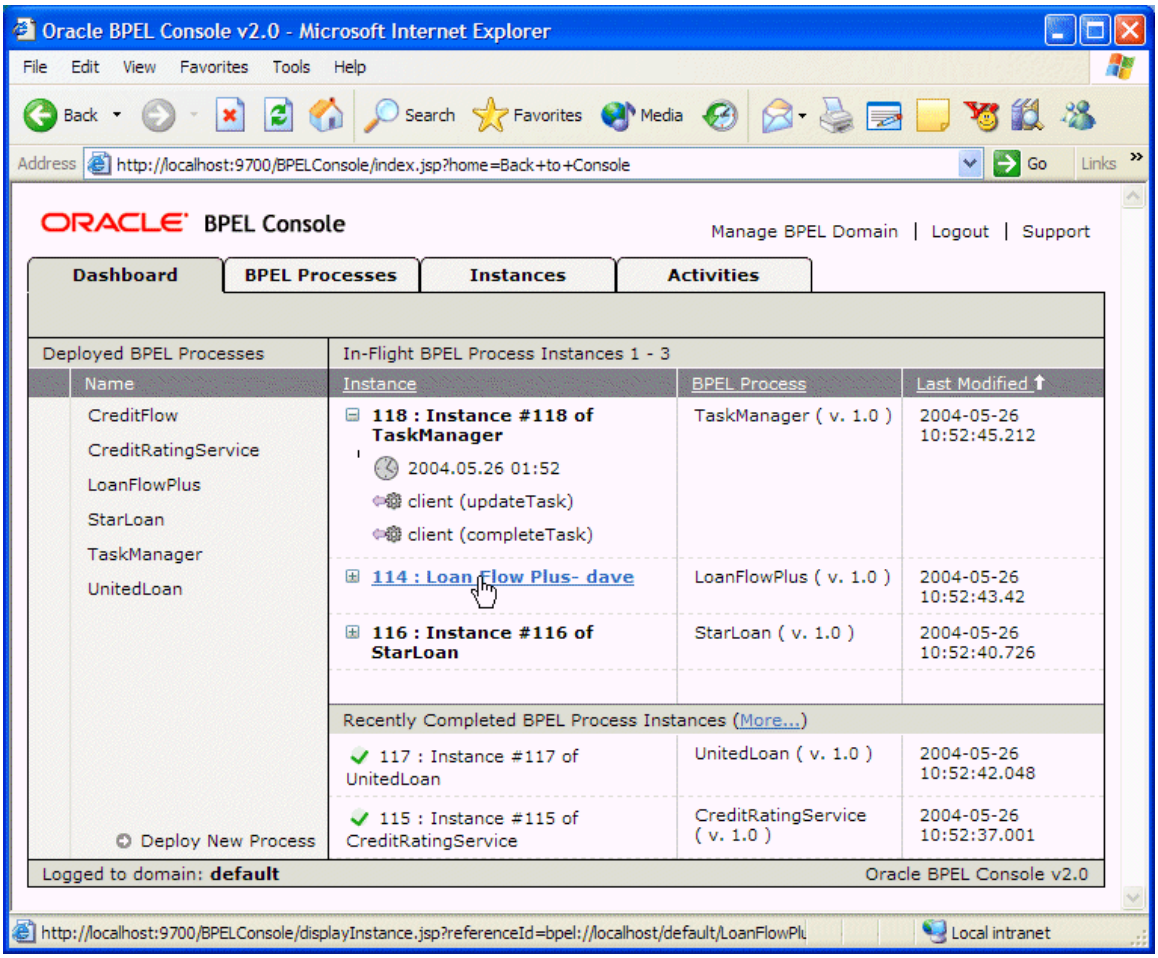
11 You will now see a web page which allows you to submit a loan application to initiate a new LoanFlowPlus instance. You can change the fields in the UI, if you choose, and click the Submit Loan Application button to initiate the new process instance:



REVIEW THE VISUAL AUDIT TRAIL

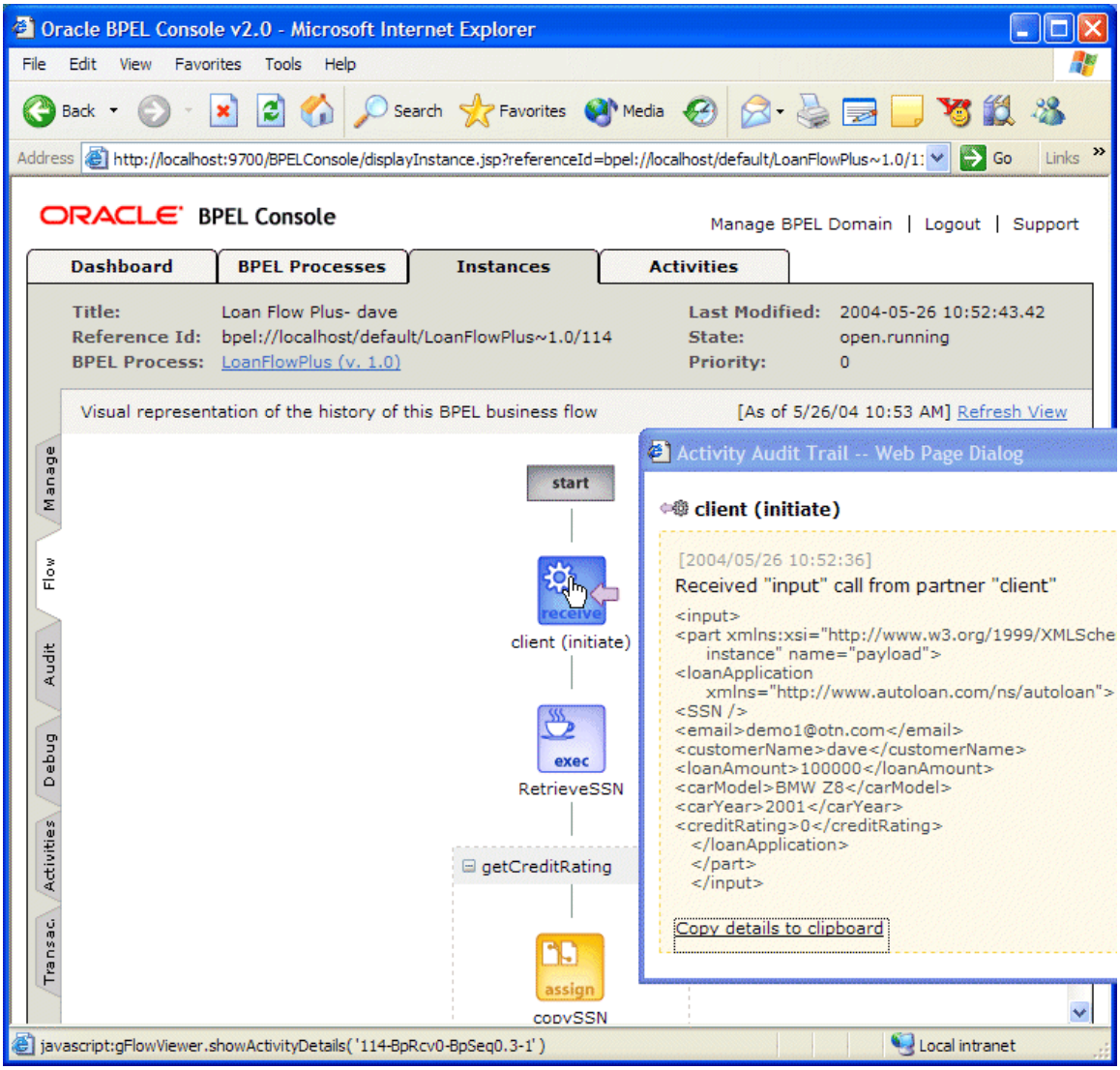
Next you will put on the hat of a developer or administrator and use the BPEL Console to view the audit trail and other status information regarding this process.

12 To do this, point a different browser window at:
http://localhost:9700/BPELConsole/
and log in if necessary. You should see several in-flight instances and several completed instances of BPEL processes. Select the active instance of the LoanFlowPlus application as shown below:

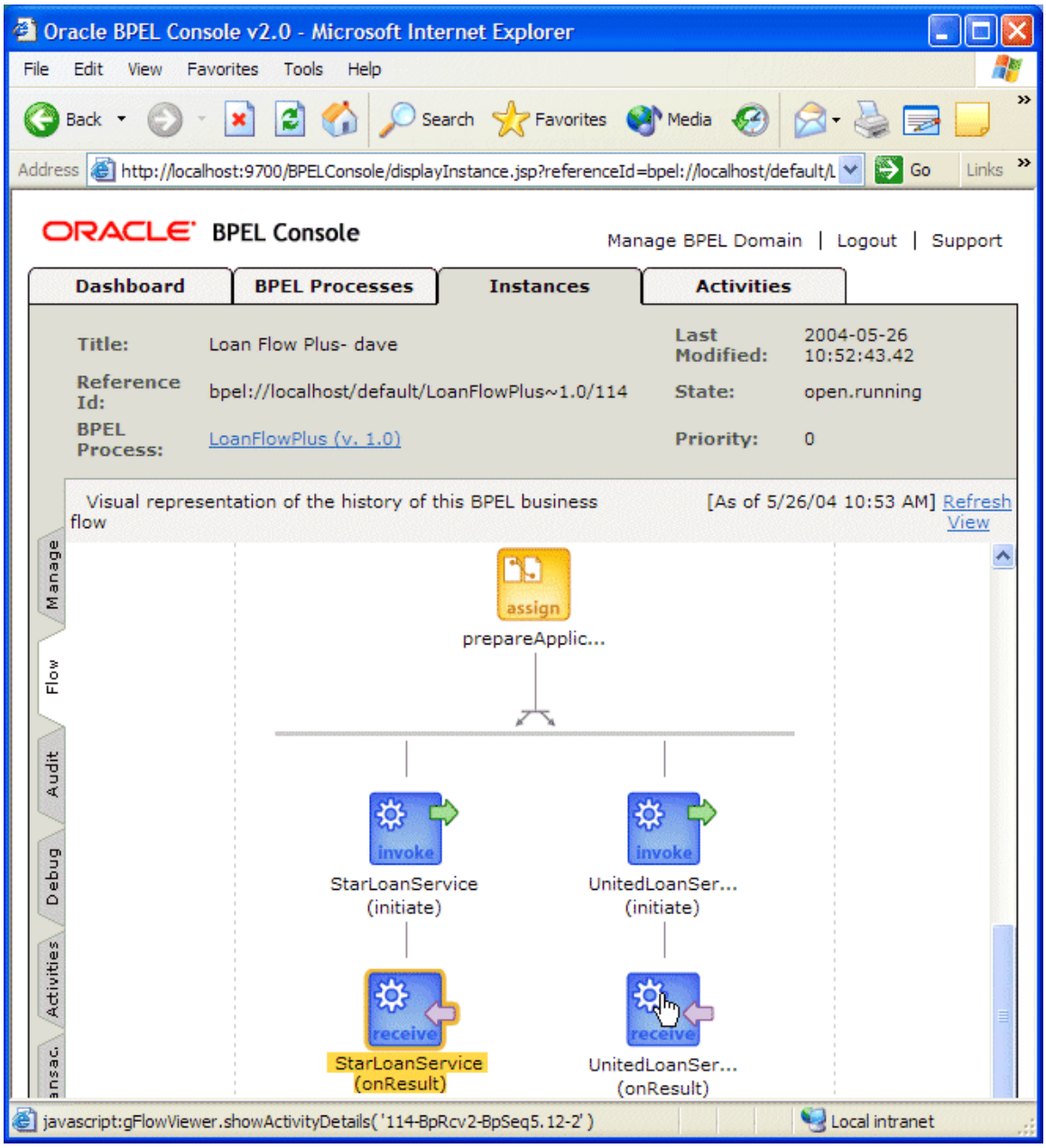


Note that everything that you see in the BPEL Console is available via an API as well. In fact, the BPEL Console itself is just a set of JSPs that can serve as a code example of how to use the BPEL Server APIs. Developers frequently use these APIs to build custom process dashboards, search screens and other interfaces so users and administrators to be able to more efficiently access and manage process instances. This is particularly common as applications go into production where there may be thousands or even millions of process instances to manage.

13 In any case, once you select the LoanFlowPlus instance in the console, you will see the "visual audit trail" which shows the current status of the process and its execution history. This audit trail is automatically generated and maintained by the BPEL Server, though developers and administrators can control what information gets logged. If you click on an activity in the visual audit trail, you will see detail information for that activity. For example, in the picture below, the client initiation has been selected and so you see the XML input message which was sent to kick off the flow.



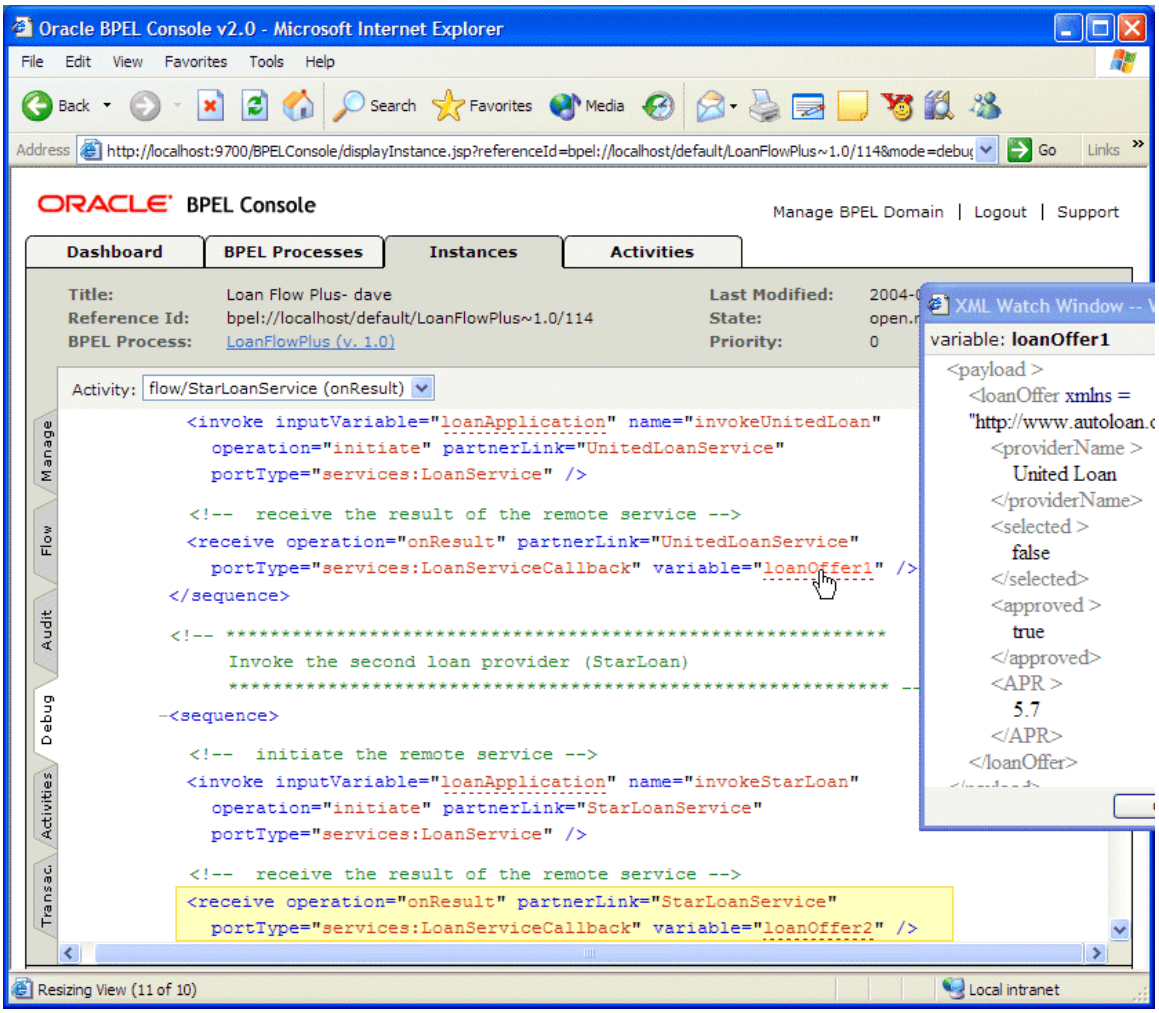
14 If you scroll down to the bottom of the visual audit trail, you will see the latest status for this instance, in this case that the StarLoan service has been initiated but the flow is waiting to receive a response (as indicated by the orange highlighting), however the UnitedLoan service has already called back with a loan offer.



15 You can also select the “Audit” tab on the left hand side of the console to see a text-based audit trail. The text audit trail includes such information as the messages exchanged with services and timestamps for when each activity was started or completed and again is both configurable and accessible via API.

DEBUGGING THE IN-FLIGHT BPEL PROCESS

16 If you now select the “Debug” tab, you will see the BPEL Debugger which takes the BPEL source that implements this process and matches it up against the state of this particular instance. Points in the code where execution is currently paused are highlighted and as you can see below, the process is currently waiting for the StarLoan service to call back with a loan offer.

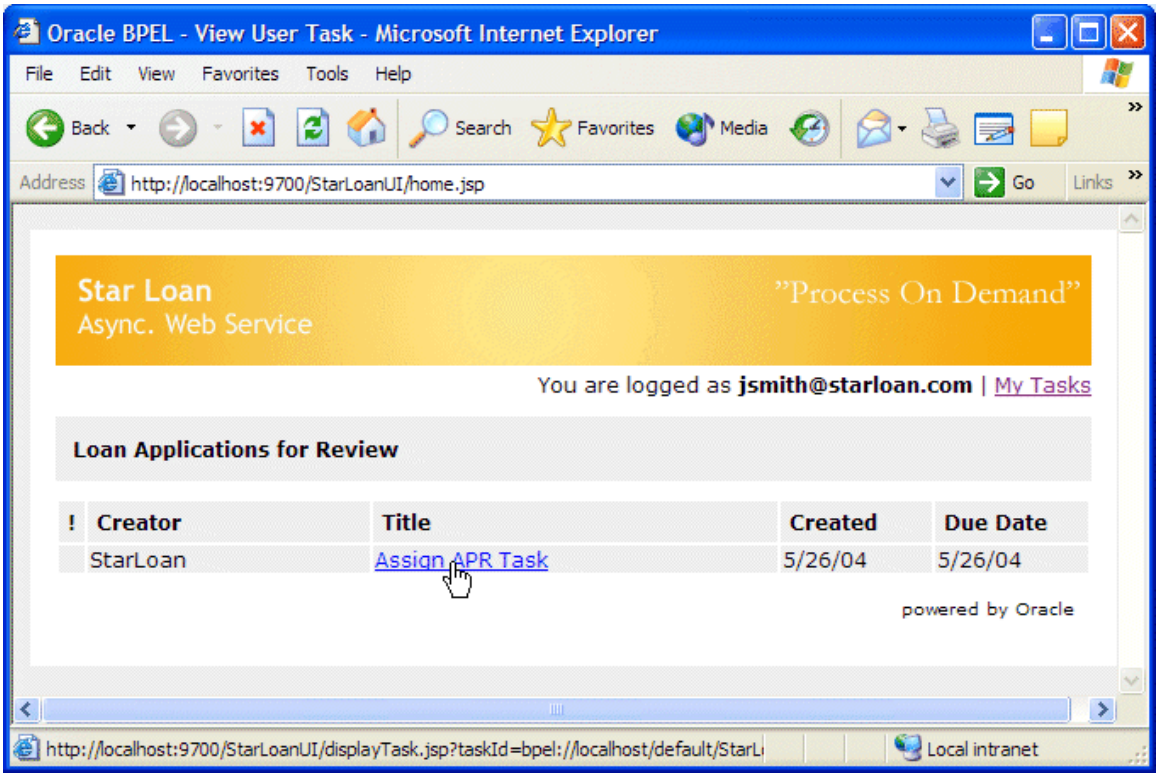


You can select variables in the debugger to inspect their current values. Shown above is the value of the loan offer variable which contains the result of the UnitedLoan service callback operation. As you can see, UnitedLoan has approved the loan application and returned a 5.7% interest rate loan offer.

COMPLETING THE ASYNCHRONOUS STARLOAN SERVICE

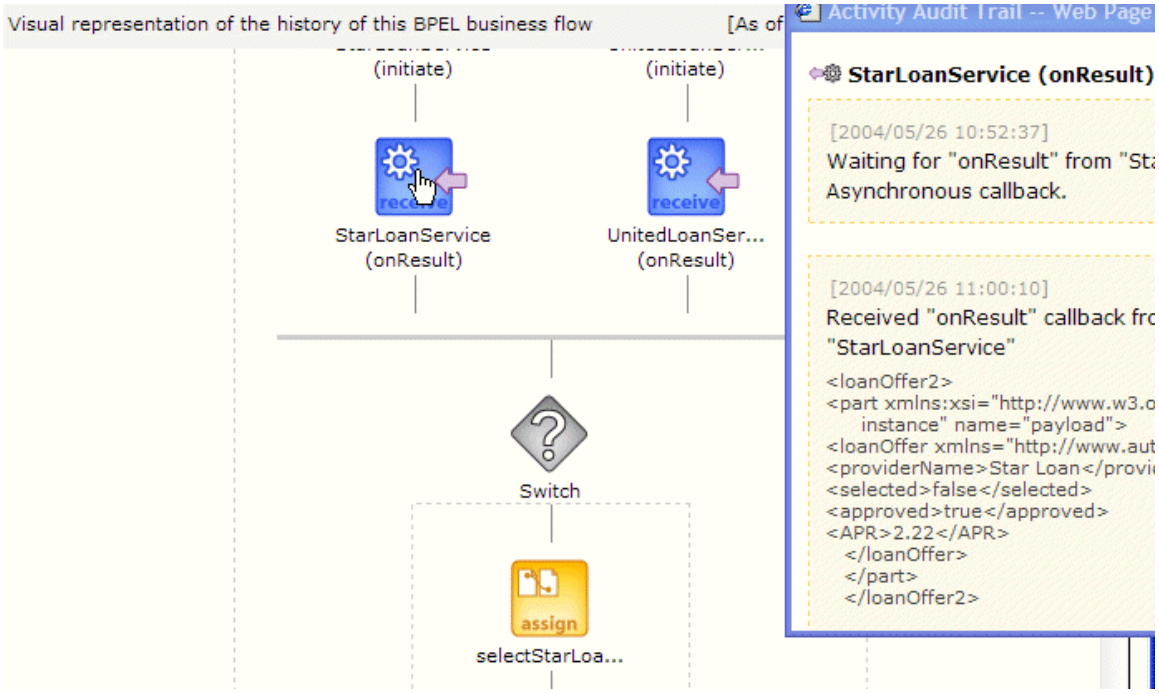
17 If you now want to complete the process, you can bring up a StarLoan customer service rep dashboard, since StarLoan requires manual processing of loan applications. Naturally, StarLoan has also been implemented as a BPEL process (though implementations are available where StarLoan is built with other Web

services toolkits, such as Microsoft .Net, Apache Axis, etc) and serves as an additional code example. To play the role of the StarLoan loan officer, point a browser at: http://localhost:9700/StarLoanUI/home.jsp



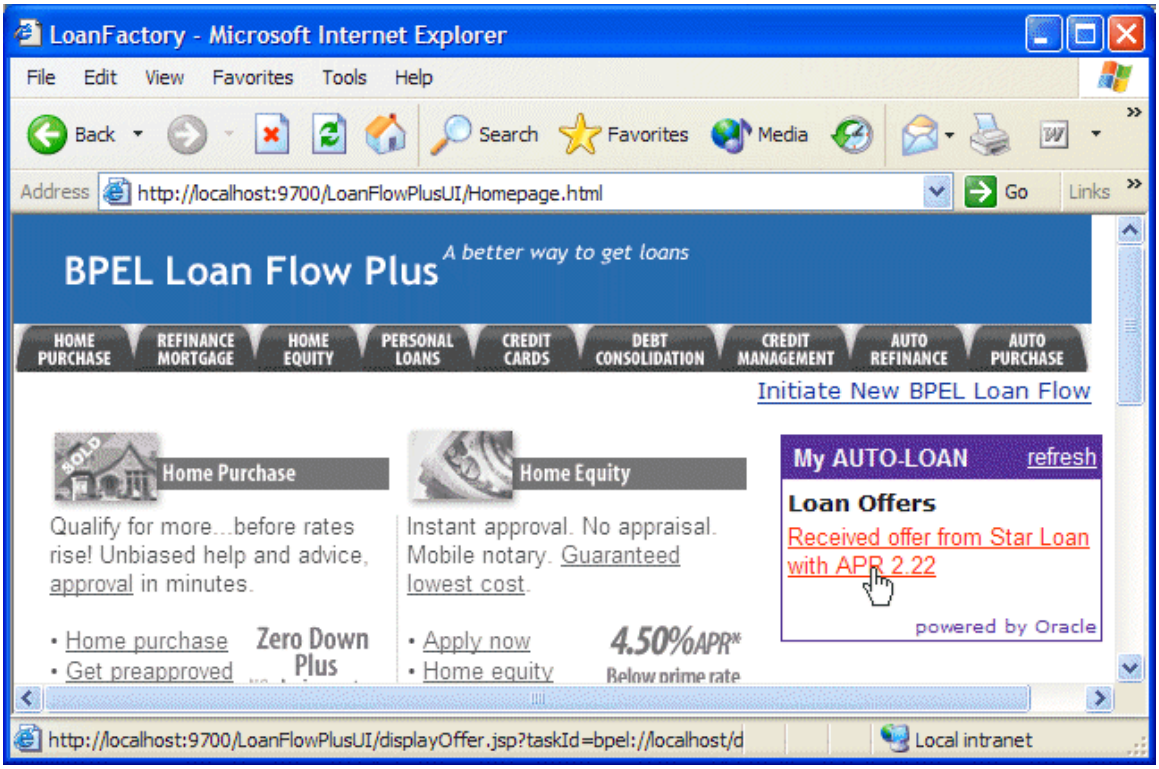
Here you should see at least one pending loan application, which you should select, enter an interest rate (go ahead and make it a nice low one as long as you are approving your own loan application...) and click the approve button. This will cause the StarLoan service to return a callback to the LoanFlowPlus process.

18 If you refresh its audit trail, you should see that it has moved along in its processing, having received the StarLoan callback with the second loan offer, selected the best loan offer and is now waiting for the customer to approve the offer.



COMPLETING A USER TASK

19 As the final step in the process, you could refresh the customer loan portal homepage where you should now see the selected loan offer waiting for review.

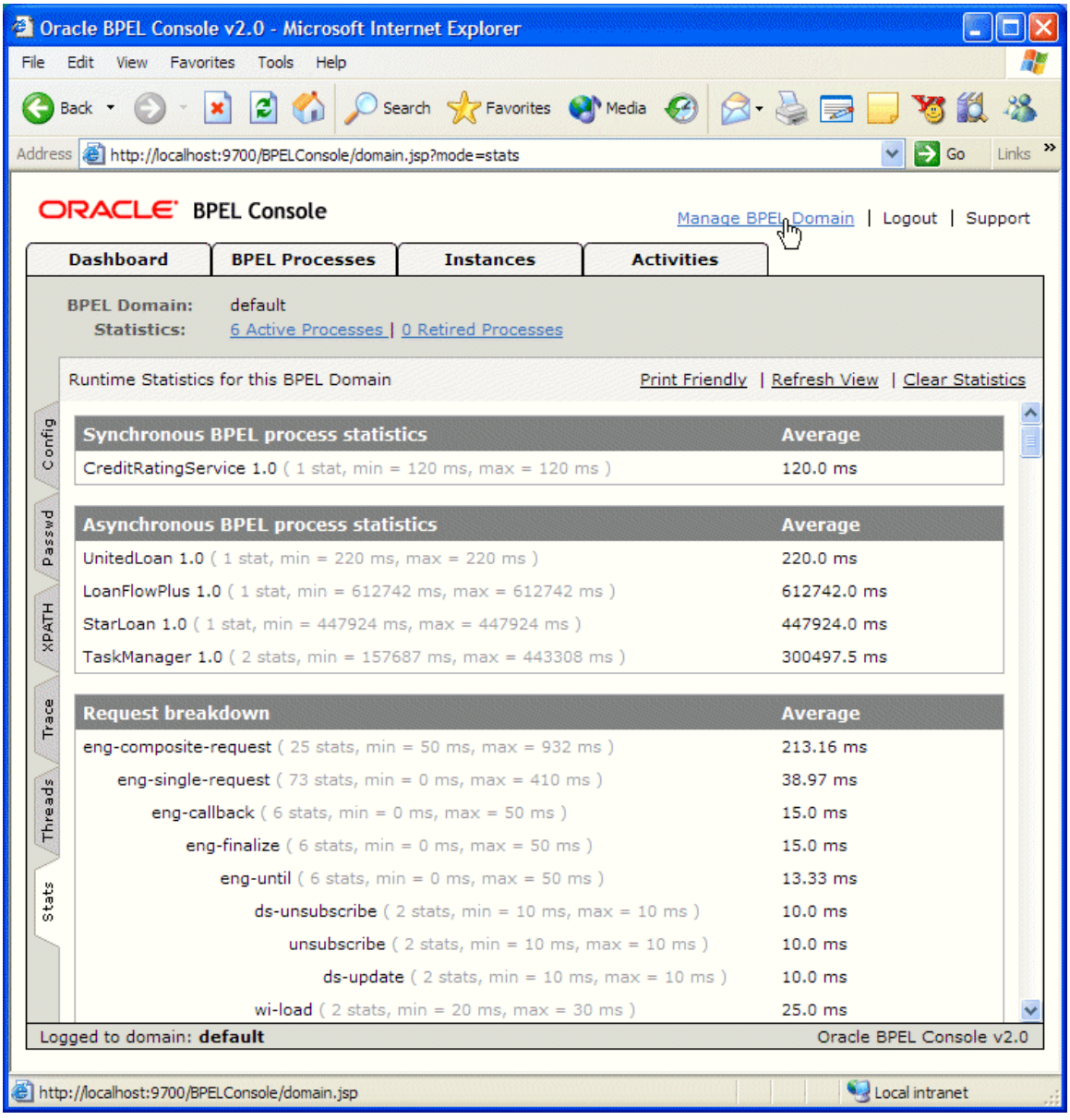


If you select this loan offer you would get a page that allows the customer to accept this offer (which would complete the instance of the LoanFlowPlus BPEL process).

PERFORMANCE TUNING

Frequently performance (including throughput and response time requirements) is a critical requirement for effectively implementing a BPEL process. Built-in to the BPEL Console is a performance tuning framework that assists developers to understand the performance of their applications and identify and resolve bottlenecks (whether in the developer's code, an external service or the BPEL Server itself).

20 You can see a page with the performance data for your completed flows by going to the BPEL Console, selecting the "Manage BPEL Domain" link in the upper right hand corner and then clicking the "Stats" tab in the lower left hand side of this page.



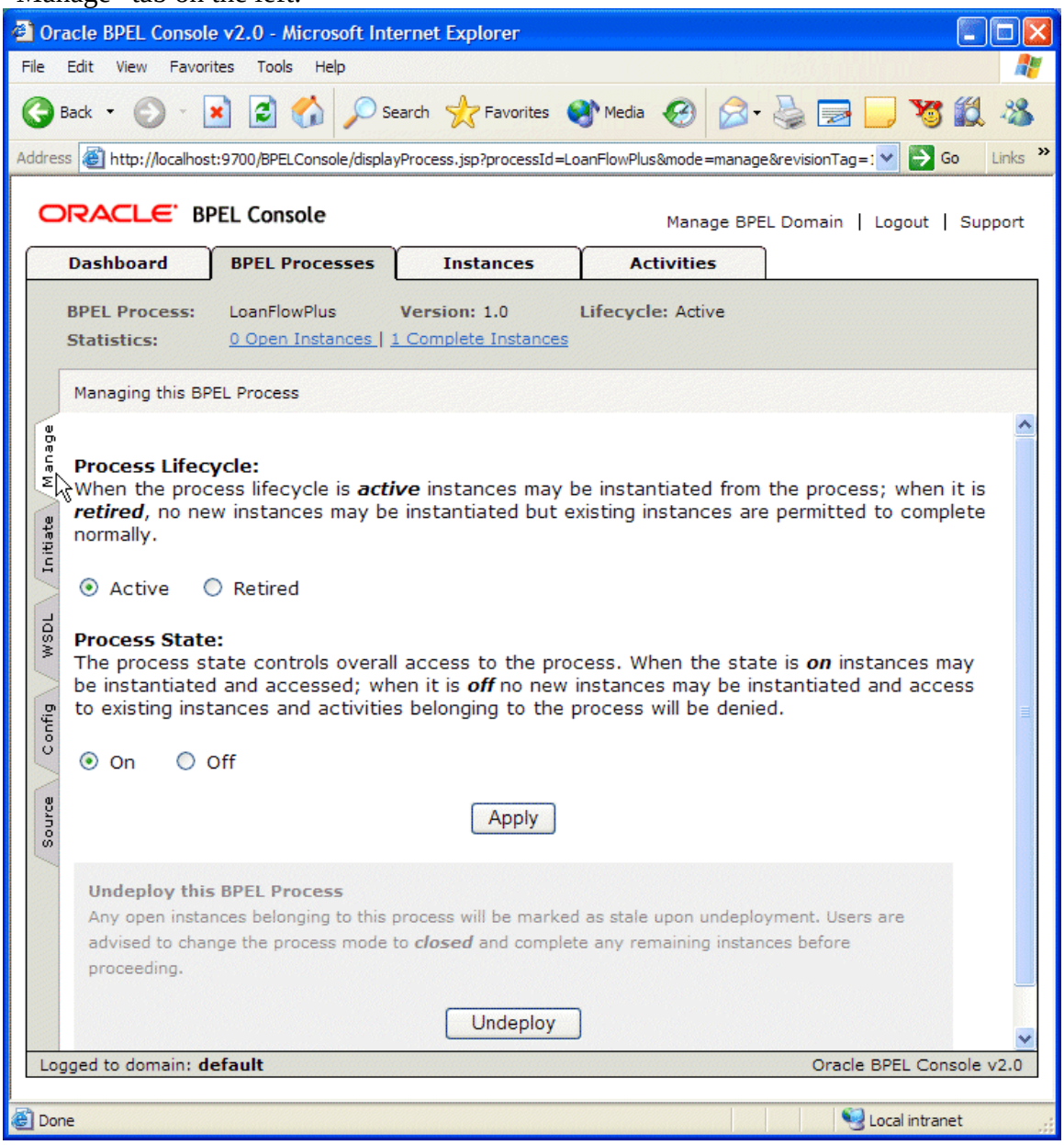
Also note that the BPEL Console allows you to test any of your deployed processes through an automatically generated HTML form interface or by passing specific XML message content to it. This test page includes stress test capability which makes it easy to do load testing of a deployed BPEL process and then view the performance statistics of the flow under stress.

PROCESS LIFECYCLE MANAGEMENT

The Oracle BPEL Process Manager and BPEL Console also include support for process lifecycle management. For example, side-by-side versioning is supported so that new implementations of a process can be "hot deployed" without disturbing current in-flight instances prior implementations of the process. In addition, the BPEL Console can be used to turn off an entire process, undeploy a process, or "retire" a process such that existing instances can complete, but new instances of the process will not be allowed.

21 This functionality is available through the process management tab. To see this tab, select the name of a deployed BPEL process in the BPEL Console and then click the

"Manage" tab on the left.



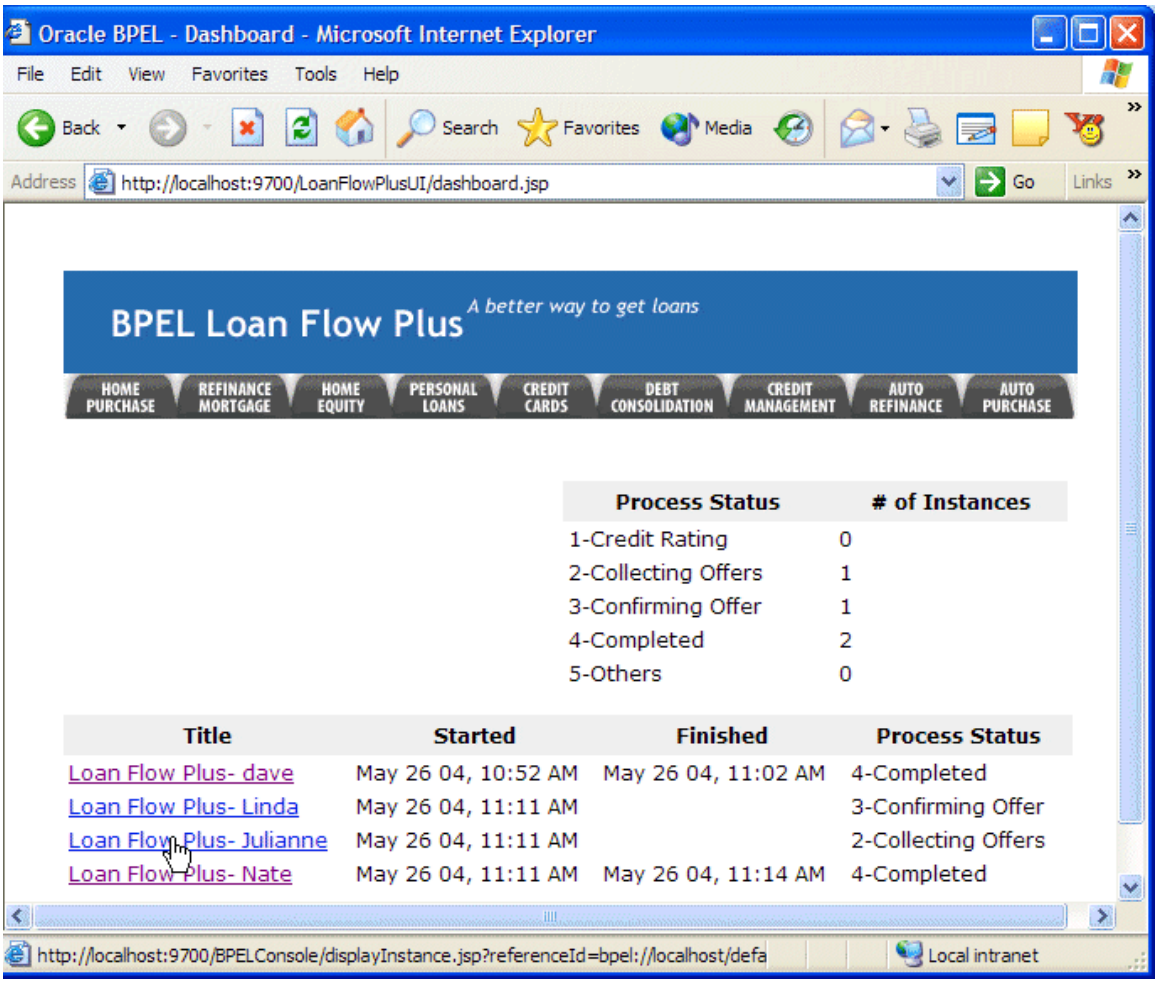
BUILD A SIMPLE REPORTING DASHBOARD

A "killer feature" of automating a business process is often to provide greater visibility into process state information. This is often realized by building a process "dashboard" which displays aggregate summary information or process instance state information. For example, an executive may want to know how many loans are currently at each stage of processing, the average time to present a loan offer to a customer, or other data based on reporting against live and competed process instances.

The Oracle BPEL Process Manager maintains a great deal of information regarding process state and, as mentioned, makes this information available via API. In addition, several reporting dashboard templates are shipped with the BPEL Server or are available

from professional services and support organizations. Such a template is included with the LoanFlowPlus process and can be accessed via the URL:

http://localhost:9700/LoanFlowPlusUI/dashboard.jsp

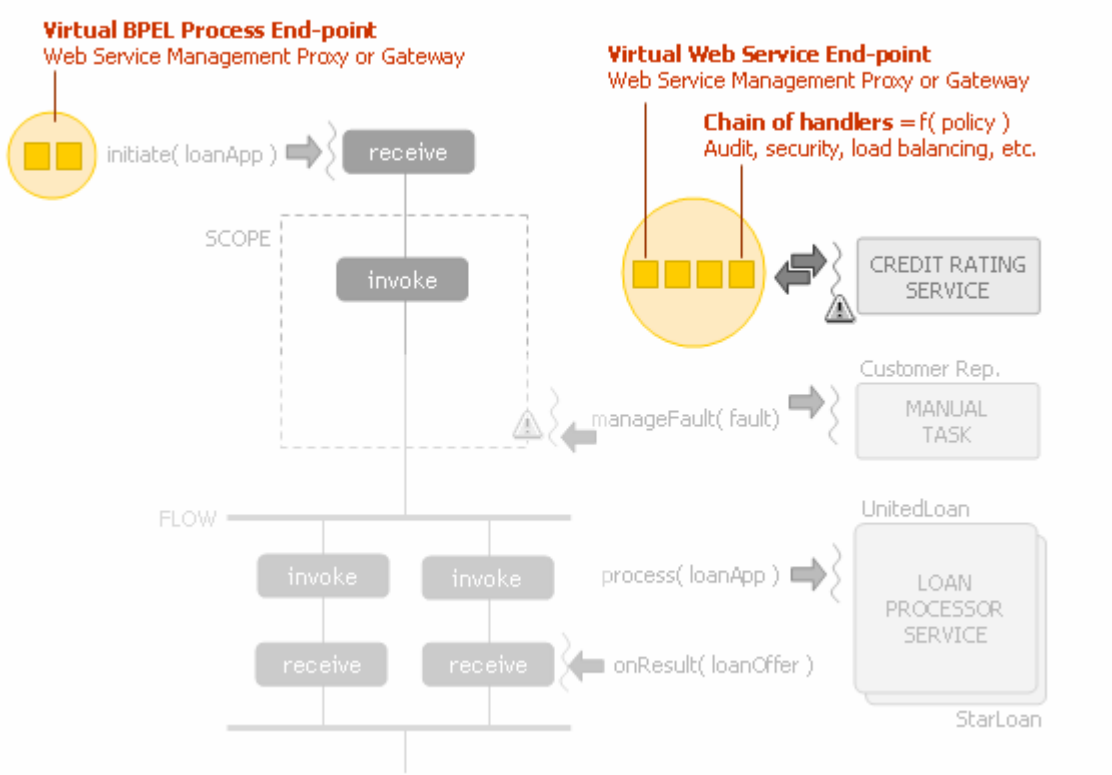


Note that the states and all of the information displayed in this report/dashboard is customizable and the templates are just intended as examples of the kind of information which is available to report on. Contact your Oracle BPEL Process Manager representative for assistance in creating reporting dashboards which are customized for your specific requirements.

INTEGRATION WITH A WEB SERVICE MANAGEMENT SOLUTION

The question frequently comes up as to the relationship between a product like the Oracle BPEL Process Manager and Web services management solutions, such as those from Computer Associates, Amberpoint and others. In both theory and practice, these technologies are highly complementary and easily integrated together. As shown in the diagram below, Web services management solutions typically provide proxies or gateways in front of Web services. This includes both “back-end” Web services which may be implemented in many different technologies as well as BPEL processes deployed

to the Oracle BPEL Server, or other BPEL servers, since a BPEL process is itself a Web service.



Here, the Web services management (WSM) solution will provide a “managed” end-point which looks to the BPEL process just like the underlying Web service. The BPEL process does not even need to know that it is connecting to a managed end-point rather than the service itself. The WSM technology then typically provides a chain of handlers so that encryption, security, load balancing and other production deployment requirements can be implemented in a uniform manner across all the Web services deployed in an enterprise. Likewise, the WSM technology can be placed logically in front of the Oracle BPEL Process Manager to allow it to manage the connections to BPEL processes. Here the only thing that is necessary is to use the Oracle BPEL Console to configure the callback port so that WS-Addressing information is sent including the WSM server’s listener port rather than the BPEL Server’s port.

Contact your local Oracle representative for information on how to integrate your selected WSM product(s) with the Oracle BPEL Server.

75+ Additional Examples

We believe that “samples are a developer’s best friends” and therefore try to continuously increase the number of sample BPEL projects which ship with the BPEL Server.

Here is a quick overview of how those samples are organized:

`C:\orabpel\sample\references\*`

Contains a BPEL project for each activity and concept defined in the BPEL language. A good way to learn about and try out a specific feature before integrating it into a larger BPEL process.

`C:\orabpel\sample\interop\*`

Contains a set of BPEL projects showcasing the interoperability of the Oracle BPEL Process Manager with Web services implemented with Microsoft .Net, Apache Axis and BEA WebLogic.

`C:\orabpel\sample\tutorials\*`

Contains a set of fairly simple BPEL processes targeting the various tasks a BPEL developer is exposed to, including: building your first BPEL process, invoking a synchronous service, invoking an asynchronous service, manipulating data elements, defining parallel branches of execution, managing faults, managing timeouts, defining correlation sets, modeling user interactions, calling Java components, manipulating XML arrays, defining complex multi-step routing, integrating XSLT and XQuery transformations, sending and receiving emails, interactions with JMS destinations and defining custom WSIF bindings.

`C:\orabpel\sample\utils\*`

Contains a set of building block services shared by the BPEL samples.

`C:\orabpel\sample\demo\*`

AmazonFlow showcases how an Amazon Web service can be integrated in a BPEL process.

CheckoutDemo showcases how to do a 2-step receive…reply…receive…reply between the client and the BPEL process.

Google Flow showcases how a Google Web service can be integrated into a BPEL process.

HotwireFlow showcases how to use correlation sets to create sophisticated and stateful interactions between a client and a BPEL process.

IBMSamples showcases how the BPEL samples shipping with BPWS4J can be executed on the Oracle BPEL Server.

LoanDemo showcases how to integrate a synchronous credit rating service and two asynchronous loan processor services into an end-to-end loan procurement application with a JSP UI to initiate the process and view loan offer results.

LoanDemoPlus is an extension to the LoanDemo sample showcasing Java embedding, exception management, including manual processing steps, and development of a richer custom user interface.

ParallelSearch showcases the ability of the BPEL server to perform parallel synchronous invocations.

ResilientDemo showcases how a BPEL process can be instrumented to properly manage run-time exceptions (see http://otn.oracle.com/bpel for more information).

SalesforceFlow showcases how the Salesforce.com sForce web services can be integrated into a BPEL process (including authentication, session management and dynamic load balancing).

TimeOffRequestDemo showcases how to model user interactions and workflow tasks within a BPEL process (see the TaskManager technote at http://otn.oracle.com/bpel for more information on the TaskManager service).